



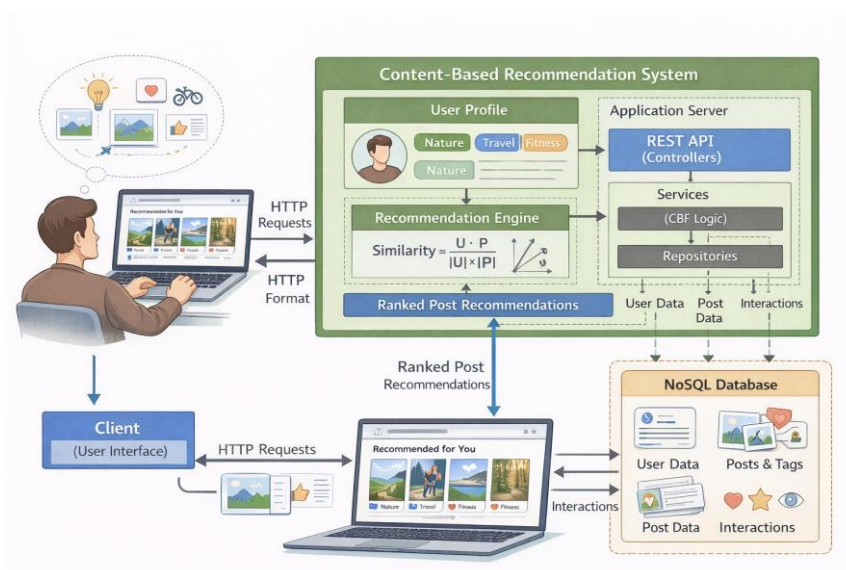
מכללת כנפי רוח - קרית נוער ירושלים

ספר פרויקט לעבודת גמר

י"ד הנדסאי תובנה (שאלון: 714918)

בנושא

גלריית מיניאטורות gallery miniatures



שם הסטודנט: יהושע נאימרק

ת"ז: 329426464

מנחה: אילן פרץ

תאריך הגשה: י"ח סיוון תשפ"ו 03/06/2026

מכללה: כנפי רוח, קרית נוער ירושלים (סמל מוסד: 140129)

תוכן עניינים

6	1. הצעת הפרויקט שאושרה ע"י משרד החינוך.....
6	1.1 תיאור נושא הפרויקט.....
6	1.2 הגדרת הבעיה האלגוריתמית.....
7	2. מבוא.....
7	2.1 הרקע לפרויקט.....
7	2.2 תהליך המחקר.....
8	2.3 רקע תיאורטי וסקירת ספרות מקצועית.....
8	2.4 אתגרים מרכזיים במחקר.....
9	2.4.1 התמודדות עם הבעיה.....
9	2.4.2 הסיבות והמוטיבציה לבחירת הנושא.....
9	2.4.3 על איזה צורך הפרויקט עונה.....
10	3. מטרות ויעדים לפיתוח המערכת.....
10	4. אתגרים ובעיות בבניית המערכת.....
10	4.1 מימוש מנגנון ההמלצות.....
11	4.2 ניהול משתמשים ו-Session.....
11	4.3 העלאת תמונות ושמירת קבצים.....
11	4.4 חיבור למסד נתונים בענן.....
12	4.5 בניית ממשק משתמש מודרני.....
12	4.6 עבודה בארכיטקטורת שכבות.....
12	4.7 עדכון מידע בזמן אמת.....
13	4.8 עבודה עם REST API.....
13	4.9 סיכום.....
13	5. הגדרת מדדי הצלחה למערכת.....
13	5.1 תקינות פונקציונלית של המערכת.....
14	5.2 הצלחת מנגנון ההמלצות.....
14	5.3 ביצועי המערכת.....
15	5.4 יציבות ואמינות.....
15	5.5 שימושיות וחווית משתמש.....
15	5.6 איכות ארכיטקטורת התוכנה.....
16	5.7 תקינות REST API.....
16	5.8 הצלחת העלאת תמונות.....
16	5.9 יכולת הרחבה עתידית.....
16	5.10 סיכום.....
17	6. תיאור פתרונות קיימים וניתוח חלופות.....
17	6.1 רשתות חברתיות כלליות.....
17	6.2 פלטפורמות אומנות ייעודיות.....

18	 Collaborative Filtering 6.3
18	 Content-Based Filtering 6.4
19	 מערכות היברידיות 6.5
19	 השוואה בין החלופות 6.6
20	 ניתוח החלופה המתאימה לפרויקט 6.7
20	 סיכום 6.8
20	 7. תיאור החלופה הנבחרת ונימוקים לבחירתה
21	 8. אפיון המערכת המוצעת
21	 8.1 ניתוח הדרישות מהמערכת
22	 8.2 מודלי המערכת
22	 8.3 אפיון פונקציונלי וביצועים עיקריים
23	 8.4 אילוצי המערכת
24	 9. תיאור הארכיטקטורה של המערכת
24	 9.1 ארכיטקטורת המערכת
26	 9.2 מודל שרת-לקוח (Client-Server Model)
26	 9.3 תהליכים של מערכת ההפעלה
26	 9.4 פרוטוקולי תקשורת
27	 9.5 שירותים חיצוניים ו-REST API
27	 9.6 סיכום הארכיטקטורה
27	 10. ניתוח תרחישים וזרימת מידע
28	 10.1 תרחיש שימוש – רישום משתמש חדש
29	 10.2 תרשימי רצף Sequence Diagram
29	 10.2.1 Login Sequence
29	 10.2.2 Create Post Sequence
29	 10.2.3 Like Post Sequence
30	 10.2.4 Personalized Feed Sequence
30	 10.3 תרחיש שימוש – צפייה בפיד מותאם אישית
31	 10.4 תרחיש שימוש – ביצוע Like
32	 10.5 תרחיש שימוש – הסרת Like
33	 10.6 תרחיש שימוש – יצירת פוסט חדש
34	 10.7 תרחיש שימוש – צפייה בפרופיל אישי
34	 10.8 תרחיש שימוש – העלאת תמונה
35	 10.9 תרחיש שימוש – שימוש ב-REST API
35	 10.10 זרימת המידע במערכת
36	 10.11 סיכום
36	 11. תיאור מסכים וממשק משתמש
37	 11.1 מסך פתיחה LandingView-

38	11.2 תיאור מסכים
38	11.2.1 RegisterView
39	11.2.2 LoginView
40	11.2.3 מסך הפיד-FeedView
40	11.2.4 מסך יצירת פוסט-CreatePostView
41	11.2.5 מסך פרופיל-ProfileView
41	11.3 עיצוב הממשק
42	11.4 ניווט בין מסכים
42	11.10 הגנת מסכים
42	11.11 סיכום
43	12. תיאור התוכנה
43	12.1 סביבת הפיתוח של התוכנה
43	12.1.1 מערכת הפעלה
43	12.1.2 סביבות הפיתוח (IDEs)
43	12.1.3 מסגרות לפיתוח התוכנה (Frameworks)
43	12.1.4 שפות תכנות
44	12.1.5 ספריות/מערכות/מנגנוני עזר נוספים
44	12.2 תיאור מבנה הפרויקט והמחלקות
44	12.2.1 עץ המודולים של הפרויקט
46	12.2.2 תרשים המחלקות UML Class Diagram
47	12.2.3 תיאור מחלקות מפורט
52	12.3 מבני נתונים בהם נעשה שימוש
53	12.4 קוד התוכנית לפעולות ואלגוריתמים עיקריים וחשובים
62	13. תיאור מסד הנתונים
62	13.1 סוג מסד הנתונים
63	13.2 מבנה בסיס הנתונים
63	13.3 אוסף המשתמשים-users
64	13.4 אוסף הפוסטים-posts
65	13.5 קשרים בין האוספים
65	13.6 Preference Vector
66	13.7 אינדקסים
67	13.8 פעולות CRUD
67	13.9 Spring Data MongoDB
67	13.10 MongoRepository
68	13.11 אבטחת מסד הנתונים
68	13.12 גיבויים ושחזור
68	13.13 יתרונות MongoDB בפרויקט

69	13.14 סיכום
69	14. מדריך למשתמש
69	14.1 כניסה למערכת
69	14.2 הרשמה למערכת
70	14.3 התחברות למערכת
70	14.4 צפייה בפייד
70	14.5 יצירת פוסט חדש
71	14.6 צפייה בפרופיל המשתמש
71	14.7 Navigation Bar
72	14.8 התנתקות מהמערכת
72	15. בדיקות והערכת המערכת
72	15.1 סביבת הבדיקות
73	15.2 סוגי הבדיקות
73	15.3 בדיקות הרשמה והתחברות
74	15.4 בדיקות יצירת פוסטים
75	15.5 בדיקות Like ו-Unlike
76	15.6 בדיקות Preference Vector
76	15.7 בדיקות מנגנון ההמלצות
77	15.8 בדיקות REST API
77	15.9 בדיקות שימושיות
77	15.10 תקלות שהתגלו במהלך הבדיקות
78	15.11 מגבלות הבדיקות
78	15.12 הערכת המערכת
78	15.13 סיכום
79	16. מסקנות
79	16.1 עמידה במטרות הפרויקט
79	16.2 יעילות מנגנון ההמלצות
80	16.3 יתרונות השימוש בטכנולוגיות שנבחרו
80	16.4 חשיבות ארכיטקטורת השכבות
81	16.5 אתגרים עיקריים במהלך הפיתוח
81	16.6 ידע וניסיון שנרכשו
82	16.7 תרומת הפרויקט
82	16.8 סיכום
82	17. פיתוחים עתידיים
83	17.1 שיפור מנגנון ההמלצות
83	17.2 מערכת תגובות
84	17.3 מערכת Followers

84	17.4 הודעות פרטיות
84	17.5 אפליקציה סלולרית
85	17.6 שיפור אבטחת המערכת
85	17.7 אחסון תמונות בענן
85	17.8 מערכת חיפוש
86	17.9 מערכת ניהול (Admin Panel)
86	17.10 שיפור חוויית המשתמש
86	17.11 פריסה לשרת ציבורי
87	17.12 סיכום
87	18. ביבליוגרפיה

1. הצעת הפרויקט שאושרה ע"י משרד החינוך

1.1 תיאור נושא הפרויקט

שם הפרויקט: Gallery Miniatures – גלריית מיניאטורות

הפרויקט עוסק בפיתוח רשת חברתית ייעודית לחובבי מיניאטורות ודיורמות. המערכת מאפשרת למשתמשים להירשם, להתחבר, לפרסם פוסטים הכוללים תמונות, כיתובים ותגיות, לבצע פעולות לייק ולצפות בפיד תוכן מותאם אישית.

המערכת פותחה כיישום Web הפועל בארכיטקטורת שרת-לקוח ומשלבת מסד נתונים בענן, העלאת תמונות וממשק משתמש מודרני.

מטרת הפרויקט היא להדגים כיצד ניתן לשלב מערכת המלצות בתוך רשת חברתית מלאה, תוך שימוש בטכנולוגיות מודרניות מתחום פיתוח ה-Web והנדסת התוכנה.

1.2 הגדרת הבעיה האלגוריתמית

אחת הבעיות המרכזיות ברשתות חברתיות היא עומס מידע, המקשה על המשתמש למצוא תכנים הרלוונטיים עבורו. ככל שמספר הפוסטים גדל, נדרש מנגנון שידע לדרג ולהציג למשתמש את התוכן המתאים ביותר לתחומי העניין שלו.

לצורך פתרון בעיה זו, פותח במערכת מנגנון המלצות המבוסס על Content-Based Filtering (CBF). האלגוריתם משתמש בתגיות של פוסטים ובונה עבור כל משתמש וקטור העדפות אישי בהתאם לפוסטים שאהב.

בעת יצירת הפיד, המערכת מחשבת ציון התאמה לכל פוסט על סמך מידת ההתאמה בין תגיות הפוסט לבין תחומי העניין של המשתמש, ומציגה את הפוסטים לפי סדר רלוונטיות.

2. מבוא

2.1 הרקע לפרויקט

בשנים האחרונות רשתות חברתיות הפכו לחלק מרכזי מחיי היום-יום של משתמשים רבים ברחבי העולם. פלטפורמות שונות מאפשרות שיתוף תמונות, סרטונים ותוכן אישי בתחומים מגוונים כגון אומנות, צילום, משחקים ותחביבים.

תחום המיניאטורות והדירמות צבר פופולריות רבה בקרב קהילות Hobby ברחבי העולם. משתמשים רבים משקיעים זמן רב ביצירת מודלים, צביעת דמויות ובניית סצנות מורכבות, אך ברוב המקרים הם משתפים את עבודותיהם בפלטפורמות כלליות שאינן מותאמות במיוחד לתחום זה. מטרת הפרויקט היא לפתח רשת חברתית ייעודית לחובבי מיניאטורות, דירמות וצביעה, המאפשרת למשתמשים להעלות פוסטים, לשתף תמונות ולקבל פיד מותאם אישית לפי תחומי העניין שלהם. המערכת כוללת מנגנון המלצות מבוסס Content-Based Filtering, אשר מתאים את הפיד לכל משתמש לפי תגיות של פוסטים שסומנו ב-Like.

2.2 תהליך המחקר

בתחילת הפרויקט בוצע מחקר על רשתות חברתיות קיימות ועל מערכות המלצה נפוצות.

נבדקו שיטות שונות להצגת תוכן מותאם אישית, כגון:

- Content-Based Filtering
- Collaborative Filtering
- Hybrid Recommendation Systems

בנוסף, בוצע מחקר על טכנולוגיות Web מודרניות לצורך פיתוח המערכת, כולל Spring Boot, MongoDB Atlas ו-Vaadin.

לאחר ניתוח החלופות הוחלט לבחור במנגנון המלצות מסוג Content-Based Filtering, מכיוון שהוא מתאים במיוחד למערכת קטנה שאינה כוללת עדיין כמות גדולה של משתמשים ונתוני אינטראקציה.

במהלך הפיתוח בוצעו בדיקות שונות למערכת, כולל בדיקות התחברות, יצירת פוסטים, העלאת תמונות, פעולות Like ויצירת פיד מותאם אישית.

2.3 רקע תיאורטי וסקירת ספרות מקצועית

מערכות המלצה הן מערכות תוכנה שמטרתן להציג למשתמש תוכן המתאים להעדפותיו האישיות.

קיימות מספר שיטות מרכזיות למערכות המלצה:

- Content-Based Filtering – שיטה המבוססת על מאפייני התוכן עצמו, כגון תגיות, קטגוריות או תיאורים.

- Collaborative Filtering – שיטה המבוססת על דמיון בין משתמשים שונים ועל התנהגות משתמשים במערכת.

- Hybrid Systems – שילוב בין מספר שיטות המלצה שונות.

בפרויקט זה נעשה שימוש בשיטת Content-Based Filtering. לכל משתמש נשמר Preference Vector אישי, המכיל תגיות ומשקלים המייצגים את תחומי העניין שלו.

כאשר המשתמש מבצע Like לפוסט, המערכת מעדכנת את משקל התגיות של אותו פוסט בתוך Preference Vector. לאחר מכן, בעת יצירת הפיד, המערכת מחשבת ציון התאמה לכל פוסט לפי מידת ההתאמה בין תגיות הפוסט לבין תחומי העניין של המשתמש.

המערכת פותחה באמצעות:

- Java

- Spring Boot

- Vaadin

- MongoDB Atlas

שילוב טכנולוגיות זה מאפשר פיתוח מערכת Web מודרנית במבנה Client-Server..

2.4 אתגרים מרכזיים במחקר

במהלך המחקר והפיתוח התמודדו מספר אתגרים מרכזיים.

האתגר הראשון היה בניית מנגנון המלצות פשוט אך יעיל, שיכול להתאים פיד אישי גם כאשר קיימת כמות קטנה יחסית של משתמשים ונתונים.

אתגר נוסף היה שילוב בין Frontend לבין Backend במסגרת מערכת Web מלאה, תוך שמירה על מבנה קוד מודולרי וברור.

בנוסף, היה צורך להתמודד עם שמירת תמונות, ניהול משתמשים, תקשורת עם MongoDB Atlas והצגת מידע בצורה נוחה למשתמש.

2.4.1 התמודדות עם הבעיה

לצורך פתרון בעיית ההמלצות נבחר להשתמש במנגנון Content-Based Filtering. המערכת משתמשת ב-Preference Vector המתעדכן לפי פעולות Like ו-Unlike של המשתמש. כל תגית מקבלת משקל בהתאם להעדפות המשתמש, וכך ניתן לחשב ציון התאמה עבור כל פוסט. בנוסף, המערכת פותחה בארכיטקטורת 3 שכבות המפרידה בין ממשק המשתמש, הלוגיקה העסקית ומסד הנתונים, דבר שאפשר פיתוח מסודר ותחזוקה נוחה יותר.

2.4.2 הסיבות והמוטיבציה לבחירת הנושא

הנושא נבחר מתוך עניין אישי בתחום המיניאטורות, הדיורמות וקהילות Hobby ברשת. בנוסף, הפרויקט משלב מספר תחומים מרכזיים במדעי המחשב:

- פיתוח Web
 - מסדי נתונים
 - מערכות המלצה
 - תכנון ארכיטקטורת תוכנה
 - עבודה עם REST API
- בחירת הנושא אפשרה לפתח מערכת מעשית הכוללת גם צד שרת, גם ממשק משתמש וגם מנגנון המלצות.

2.4.3 על איזה צורך הפרויקט עונה

הפרויקט נותן מענה לצורך בפלטפורמה ייעודית לחובבי מיניאטורות ודיורמות. רוב הפלטפורמות הקיימות הן רשתות חברתיות כלליות שאינן מותאמות במיוחד לקהילת Hobby. זו.

המערכת מאפשרת למשתמשים:

- לשתף עבודות ופרויקטים
 - לצפות בפוסטים של משתמשים אחרים
 - לקבל פיד מותאם אישית
 - לנהל פרופיל אישי
 - לבצע אינטראקציה עם פוסטים באמצעות Like
- בנוסף, המערכת מאפשרת גם למשתמשי Guest לצפות בפיד הציבורי, דבר המשפר את נגישות המערכת ואת חוויית המשתמש.

3. מטרות ויעדים לפיתוח המערכת

מטרת הפרויקט היא לפתח רשת חברתית ייעודית לחובבי מיניאטורות, דיורמות ופרויקטי צביעה, הכוללת מנגנון המלצות מותאם אישית.

המערכת נועדה לאפשר למשתמשים לשתף תמונות של עבודותיהם, לצפות בפוסטים של משתמשים אחרים ולקבל פיד מותאם אישית לפי תחומי העניין שלהם.

יעדי המערכת הם:

- פיתוח מערכת Web מודרנית המבוססת על ארכיטקטורת Client-Server.
- יצירת מערכת משתמשים הכוללת הרשמה, התחברות והתנתקות.
- פיתוח מנגנון העלאת פוסטים הכוללים תמונות, תגיות ותיאור טקסטואלי.
- פיתוח מנגנון Like ו-Unlike לצורך אינטראקציה עם פוסטים.
- פיתוח מנגנון המלצות מבוסס Content-Based Filtering.
- יצירת Preference Vector אישי לכל משתמש לפי פעולות Like שביצע.
- פיתוח פיד מותאם אישית המציג פוסטים לפי רמת התאמה למשתמש.
- שמירת הנתונים במסד נתונים MongoDB Atlas בענן.
- פיתוח ממשק משתמש ידידותי ונוח באמצעות Vaadin.
- יצירת תמיכה במשתמשי Guest, המאפשרת צפייה בפיד הציבורי ללא התחברות.
- שמירה על מבנה קוד מודולרי המאפשר תחזוקה והרחבה עתידית של המערכת.

4. אתגרים ובעיות בבניית המערכת

במהלך פיתוח המערכת התגלו מספר אתגרים טכנולוגיים והנדסיים אשר דרשו תכנון נכון, פתרונות תוכנה מתאימים והתמודדות עם בעיות שונות הקשורות לפיתוח מערכות Web מודרניות.

המערכת משלבת מספר תחומים שונים, ביניהם פיתוח ממשק משתמש, ניהול מסד נתונים, תקשורת רשת, העלאת קבצים ומנגנון המלצות מבוסס אלגוריתם. שילוב זה יצר אתגרים במספר רמות.

4.1 מימוש מנגנון ההמלצות

אחד האתגרים המרכזיים בפרויקט היה פיתוח מנגנון המלצות המבוסס על Content-Based Filtering (CBF).

האתגר המרכזי היה למצוא דרך לייצג את תחומי העניין של המשתמש בצורה דינמית, כך שהמערכת תוכל ללמוד אילו סוגי פוסטים מעניינים אותו.

לצורך כך פותח Preference Vector אישי לכל משתמש. וקטור זה מתעדכן בהתאם לפעולות Like ו-Unlike שביצע המשתמש, ומשמש לחישוב ציון התאמה עבור כל פוסט. בנוסף, היה צורך למיין את הפיד בזמן אמת בהתאם לציוני ההתאמה של הפוסטים.

4.2 ניהול משתמשים ו-Session

אתגר נוסף היה ניהול המשתמש המחובר במערכת.

המערכת נדרשה:

- לזהות האם משתמש מחובר
 - למנוע גישה למסכים פרטיים ללא התחברות
 - לשמור את פרטי המשתמש בזמן העבודה במערכת
- לצורך כך פותחה מחלקת SessionManager האחראית לניהול המשתמש המחובר ולשמירת המידע במהלך העבודה.

4.3 העלאת תמונות ושמירת קבצים

המערכת מאפשרת למשתמשים להעלות תמונות מהמחשב האישי ולצרף אותן לפוסטים.

האתגר כלל:

- קבלת קבצים מהדפדפן
 - שמירתם בתיקייה מקומית
 - יצירת כתובת URL תקינה
 - הצגת התמונות בפיד ובפרופיל
- לצורך פתרון הבעיה פותח FileStorageService ונעשה שימוש ב-WebConfig לצורך מיפוי תיקיית uploads.

4.4 חיבור למסד נתונים בענן

המערכת משתמשת ב-MongoDB Atlas הפועל בענן.

האתגרים כללו:

- יצירת חיבור מאובטח למסד הנתונים
- שמירת נתונים בצורה תקינה
- עבודה עם Collections ו-Documents

- טיפול בחיבורי רשת ובעיות תקשורת
- בנוסף, היה צורך ללמוד עבודה עם Spring Data MongoDB ועם MongoRepository.

4.5 בניית ממשק משתמש מודרני

בניית ממשק משתמש אינטראקטיבי הייתה אתגר משמעותי נוסף.

הממשק נדרש להיות:

- ברור ונוח לשימוש
- מותאם להצגת תמונות
- בנוי בצורה מודולרית
- בעל ניווט פשוט בין מסכים
- לצורך כך נעשה שימוש ב-Vaadin ובקומפוננטות כגון:

VerticalLayout •

HorizontalLayout •

Button •

Image •

Upload •

4.6 עבודה בארכיטקטורת שכבות

המערכת פותחה בארכיטקטורת שכבות הכוללת:

UI Layer •

Service Layer •

Repository Layer •

Database Layer •

האתגר היה להפריד בצורה נכונה בין אחריות הרכיבים השונים ולמנוע תלות ישירה בין שכבות. גישה זו דרשה תכנון נכון של המחלקות והשירותים במערכת.

4.7 עדכון מידע בזמן אמת

כאשר המשתמש ביצע Like או Unlike, היה צורך:

- לעדכן את מסד הנתונים

- לעדכן את Preference Vector

- לחשב מחדש את הפיד

- לעדכן את הממשק

האתגר היה לוודא שכל הנתונים נשמרים בצורה עקבית ושינויים מוצגים מיד למשתמש.

4.8 עבודה עם REST API

פיתוח REST API דרש:

- יצירת Controllers

- החזרת נתונים בפורמט JSON

- תמיכה בבקשות HTTP

- בדיקות באמצעות Postman

האתגר היה לבצע אינטגרציה מלאה בין שכבת השירותים לבין ה-API.

4.9 סיכום

פיתוח המערכת כלל התמודדות עם מגוון אתגרים מתחומי פיתוח ה-Web, מסדי הנתונים, ניהול משתמשים ומערכות המלצה. פתרון אתגרים אלו אפשר בניית מערכת יציבה, מודולרית ומתפקדת, תוך יישום עקרונות הנדסת תוכנה וטכנולוגיות מודרניות.

5. הגדרת מדדי הצלחה למערכת

לאחר הגדרת מטרות הפרויקט והאתגרים המרכזיים בפיתוח המערכת, הוגדרו מדדי הצלחה שנועדו לבדוק האם המערכת עומדת בדרישות שהוצבו בתחילת העבודה.

מדדי ההצלחה מאפשרים להעריך את איכות המערכת מבחינה פונקציונלית, טכנולוגית וחווית משתמש.

5.1 תקינות פונקציונלית של המערכת

המדד הראשון להצלחת המערכת הוא תקינות הפעולות המרכזיות.

המערכת נדרשה לבצע בצורה תקינה את הפעולות הבאות:

- הרשמת משתמשים חדשים

- התחברות למערכת

- יצירת פוסטים

- העלאת תמונות
 - ביצוע Like ו-Unlike
 - הצגת פיד מותאם אישית
 - צפייה בפרופיל אישי
 - שמירת נתונים במסד הנתונים
- המערכת תיחשב מוצלחת אם כל הפעולות יתבצעו ללא שגיאות ותוך שמירה נכונה של הנתונים.

5.2 הצלחת מנגנון ההמלצות

אחד היעדים המרכזיים של הפרויקט הוא מימוש מנגנון המלצות מבוסס Content-Based Filtering.

המערכת תיחשב מוצלחת אם:

- פוסטים רלוונטיים יוצגו גבוה יותר בפיד
 - Like ישפיע על העדפות המשתמש
 - Unlike יעדכן את Preference Vector
 - הפיד ישתנה בהתאם לפעילות המשתמש
- בנוסף, נדרש שמנגנון ההמלצות יהיה:
- יציב
 - ברור להבנה
 - ניתן להסבר וניתוח

5.3 ביצועי המערכת

המערכת נדרשה להגיב בזמן סביר לפעולות המשתמש.

מדדי הביצועים שנבדקו:

- זמן טעינת הפיד
- זמן התחברות
- זמן יצירת פוסט
- זמן העלאת תמונה
- זמן תגובה של REST API

המערכת נחשבת תקינה אם זמן התגובה קצר ואינו פוגע בחוויית המשתמש.

5.4 יציבות ואמינות

המערכת נדרשה לפעול בצורה יציבה לאורך זמן ולשמור על עקביות הנתונים.

הצלחה בתחום זה נמדדה לפי:

- שמירה תקינה של נתונים במסד הנתונים
- מניעת קריסות
- טיפול תקין בשגיאות
- שמירה עקבית של Session המשתמש

5.5 שימושיות וחויית משתמש

הממשק נדרש להיות:

- ברור
- נוח לשימוש
- אינטואיטיבי
- פשוט לניווט

המערכת תיחשב מוצלחת אם משתמשים יכולים לבצע פעולות ללא ידע טכני מתקדם.

5.6 איכות ארכיטקטורת התוכנה

המערכת פותחה בארכיטקטורת שכבות.

מדדי הצלחה בתחום זה:

- הפרדה בין שכבות המערכת
- קוד מודולרי
- תחזוקה פשוטה
- אפשרות להרחבה עתידית

5.7 תקינות REST API

המערכת כוללת REST API.

ה-API ייחשב תקין אם:

- הבקשות מחזירות JSON תקין
- הנתונים נשמרים ונשלפים בהצלחה
- Endpoints פועלים ללא שגיאות

5.8 הצלחת העלאת תמונות

המערכת מאפשרת העלאת קבצים מהמחשב המקומי.

המערכת תיחשב מוצלחת אם:

- התמונות נשמרות בהצלחה
- הקבצים מוצגים בפיד
- הנתונים נשמרים בצורה תקינה

5.9 יכולת הרחבה עתידית

אחד ממדדי ההצלחה היה פיתוח מערכת הניתנת להרחבה.

המערכת נדרשה לאפשר בעתיד:

- הוספת מערכת תגובות
- הוספת Followers
- פיתוח אפליקציה סלולרית
- שילוב אלגוריתמים מתקדמים יותר

5.10 סיכום

מדדי ההצלחה שהוגדרו בפרויקט אפשרו לבדוק את איכות המערכת מבחינה פונקציונלית, טכנולוגית וחווית משתמש. הצלחת המערכת נמדדה לא רק לפי תקינות הפעולות הבסיסיות, אלא גם לפי איכות מנגנון ההמלצות, יציבות המערכת, נוחות השימוש ויכולת ההרחבה העתידית שלה.

6. תיאור פתרונות קיימים וניתוח חלופות

בתחום הרשתות החברתיות ושיתוף התוכן החזותי קיימים פתרונות רבים המאפשרים למשתמשים לפרסם תמונות, לצפות בתוכן ולקבל המלצות מותאמות אישית. מערכות אלו משתמשות בטכנולוגיות שונות ובמנגנוני המלצה מתקדמים לצורך התאמת תוכן למשתמש.

במסגרת הפרויקט נבחנו מספר פתרונות וגישות אפשריות לצורך פיתוח מערכת ההמלצות ומבנה המערכת.

6.1 רשתות חברתיות כלליות

רשתות חברתיות גדולות כגון Instagram ו-Pinterest מאפשרות שיתוף תמונות, סרטונים ותוכן חזותי. מערכות אלו כוללות:

- פיד מותאם אישית
- מנגנוני המלצה
- מערכת לייקים
- תגובות ושיתוף תוכן
- ### יתרונות
- בסיס משתמשים גדול
- אלגוריתמים מתקדמים
- חוויית משתמש עשירה

- ### חסרונות
- עומס מידע גדול
- חוסר מיקוד בתחום ייעודי
- מורכבות גבוהה
- חוסר שקיפות במנגנון ההמלצות

6.2 פלטפורמות אומנות ייעודיות

פלטפורמות כגון ArtStation ו-DeviantArt מתמקדות בתוכן אומנותי ויצירתי. ### יתרונות

- קהל יעד ממוקד

- תוכן איכותי בתחום האומנות
- סיווג תוכן לפי קטגוריות
- ### חסרונות
- פחות התאמה אישית למשתמש
- פחות דגש על מנגנוני המלצה
- תלות בפופולריות של יוצרים

Collaborative Filtering 6.3

Collaborative Filtering הוא מנגנון המלצות המבוסס על דמיון בין משתמשים.

המערכת בודקת אילו משתמשים מתנהגים בצורה דומה וממליצה על תכנים שאהבו משתמשים אחרים בעלי תחומי עניין דומים.

יתרונות

- מסוגל לגלות קשרים חבויים בין משתמשים
- מספק המלצות מגוונות
- נפוץ במערכות מסחריות גדולות

חסרונות

- דורש כמות גדולה של משתמשים ונתונים
- בעיית Cold Start למשתמשים חדשים
- מורכבות חישובית גבוהה

Content-Based Filtering 6.4

גישה זו מבוססת על ניתוח מאפייני התוכן עצמו.

המערכת בודקת תגיות, קטגוריות ומאפיינים של פוסטים ומשווה אותם להעדפות המשתמש.

יתרונות

- אינו תלוי במשתמשים אחרים
- מתאים למערכות קטנות
- שקוף וקל להסבר

- פשוט יחסית למימוש
- ### חסרונות
- תלוי באיכות התגיות
- עלול להציג תוכן דומה מדי
- פחות מגוון מפתרונות אחרים

6.5 מערכות היברידיות

מערכות Hybrid משלבות מספר שיטות המלצה יחד, לדוגמה:

- Collaborative Filtering
- Content-Based Filtering
- Machine Learning
- ### יתרונות
- דיוק גבוה יותר
- המלצות מגוונות יותר
- התאמה אישית מתקדמת
- ### חסרונות
- מורכבות פיתוח גבוהה
- דרישות חומרה גבוהות
- צורך בכמות נתונים גדולה

6.6 השוואה בין החלופות

פתרון	יתרונות	חסרונות
רשתות חברתיות כלליות	התאמה אישית מתקדמת	עומס מידע וחוסר מיקוד
פלטפורמות אומנות	קהל יעד ממוקד	פחות התאמה אישית
Collaborative Filtering	לומד מדפוסי משתמשים	דורש הרבה נתונים
Content-Based Filtering	פשוט ושקוף	פחות מגוון
Hybrid Systems	דיוק גבוה	מורכבות גבוהה

6.7 ניתוח החלופה המתאימה לפרויקט

לאחר בחינת החלופות השונות, נמצא כי Content-Based Filtering הוא הפתרון המתאים ביותר לפרויקט זה.

הסיבות לבחירה:

- המערכת נמצאת בשלב התחלתי עם מספר משתמשים קטן
 - נדרש פתרון ברור וקל להסבר
 - אין צורך במודלי Machine Learning מורכבים
 - ניתן לממש את האלגוריתם בצורה פשוטה ויציבה
 - מתאים להיקף ולמטרות הפרויקט
- בנוסף, האלגוריתם מאפשר להציג בצורה ברורה כיצד המערכת מחשבת התאמה בין משתמשים לפוסטים.

6.8 סיכום

במסגרת הפרויקט נבחנו מספר פתרונות וטכנולוגיות מתחום מערכות ההמלצה והרשתות החברתיות. לאחר ניתוח היתרונות והחסרונות של כל גישה, נבחרה שיטת Content-Based Filtering בשל פשטותה, יציבותה והתאמתה למערכת קטנה וייעודית. בחירה זו אפשרה פיתוח מנגנון המלצות ברור, יעיל וניתן להסבר, התואם את מטרות הפרויקט.

7. תיאור החלופה הנבחרת ונימוקים לבחירתה

לאחר ניתוח החלופות השונות בתחום מערכות ההמלצה והרשתות החברתיות, נבחר לפתח מערכת רשת חברתית המבוססת על מנגנון המלצות מסוג Content-Based Filtering (CBF).

החלופה נבחרה מפני שהיא מתאימה במיוחד למערכת קטנה וייעודית, שאינה כוללת עדיין כמות גדולה של משתמשים או מידע היסטורי. בניגוד ל-Collaborative Filtering, שדורש נתונים רבים על התנהגות משתמשים שונים, שיטת Content-Based Filtering יכולה לפעול כבר בשלבים הראשונים של המערכת על בסיס תגיות הפוסטים ופעולות Like של המשתמש.

בפתרון שנבחר, לכל משתמש נשמר Preference Vector אישי, המכיל תגיות ומשקלים המייצגים את תחומי העניין שלו. כאשר המשתמש מבצע Like לפוסט, המערכת מעדכנת את המשקלים של תגיות הפוסט בתוך וקטור ההעדפות. כאשר המשתמש מבצע Unlike, ההעדפות מתעדכנות מחדש בהתאם.

בעת יצירת הפיד, המערכת מחשבת לכל פוסט ציון התאמה לפי מידת ההתאמה בין תגיות הפוסט לבין Preference Vector של המשתמש. פוסטים בעלי ציון גבוה יותר מוצגים גבוה יותר בפיד.

המערכת שפותחה משלבת ממשק משתמש מבוסס Vaadin, צד שרת המבוסס על Spring Boot, מסד נתונים בענן מסוג MongoDB Atlas ושכבת שירותים הכוללת את הלוגיקה העסקית ואת מנגנון ההמלצות.

הבחירה בפתרון זה מספקת מספר יתרונות מרכזיים: האלגוריתם פשוט יחסית להבנה ולמימוש, ניתן להסביר בקלות את אופן פעולתו, הוא מתאים להיקף הפרויקט, ואפשר להרחיב אותו בעתיד באמצעות שיטות המלצה מתקדמות יותר.

לכן, החלופה שנבחרה עונה בצורה טובה על מטרות הפרויקט: פיתוח רשת חברתית ייעודית עם פיד מותאם אישית, שמירה על מבנה תוכנה מודולרי ויכולת הרחבה עתידית.

8. אפיון המערכת המוצעת

8.1 ניתוח הדרישות מהמערכת

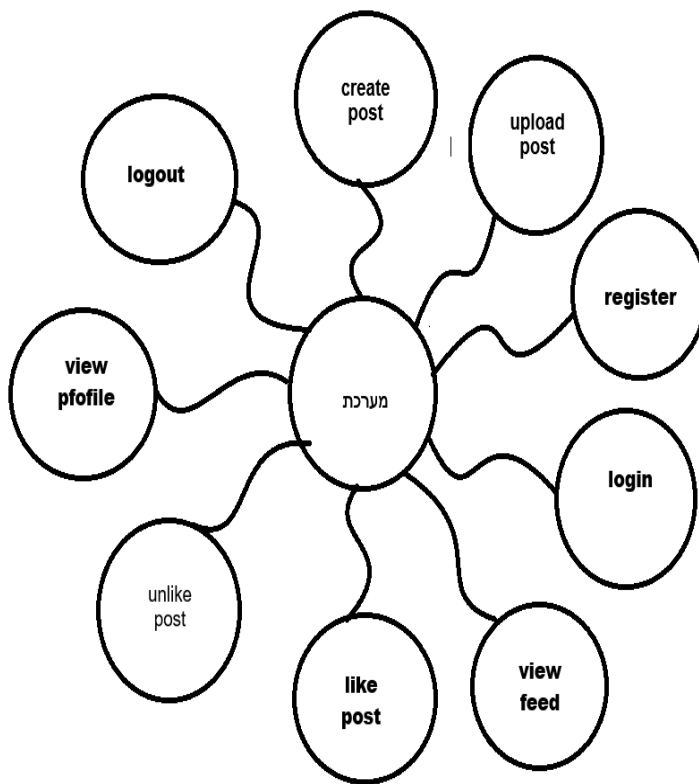
המערכת פותחה כרשת חברתית לשיתוף תמונות של מיניאטורות, דיורמות ופרויקטי צביעה, הכוללת מנגנון המלצות מותאם אישית.

הדרישות המרכזיות מהמערכת הן:

- ניהול משתמשים – המערכת צריכה לאפשר הרשמה, התחברות והתנתקות של משתמשים.
- תמיכה ב-Guest Mode – משתמשים שאינם מחוברים יכולים לצפות בפיד הציבורי, אך אינם יכולים לבצע פעולות הדורשות הרשאות משתמש.
- ניהול פוסטים – משתמשים מחוברים יכולים ליצור פוסטים חדשים הכוללים תמונה, כיתוב ותגיות.
- מערכת לייקים – משתמשים יכולים לבצע Like ו-Unlike לפוסטים.
- מנגנון המלצות – המערכת צריכה להציג פיד מותאם אישית לפי תחומי העניין של המשתמש.
- שמירת נתונים – כל נתוני המשתמשים והפוסטים נשמרים במסד הנתונים MongoDB Atlas.
- ממשק משתמש ידידותי – המערכת צריכה לכלול ממשק ברור ונוח לשימוש.
- עבודה בסביבת Web – המערכת צריכה לפעול דרך דפדפן אינטרנט ללא צורך בהתקנת תוכנה נוספת.
- תמיכה בהעלאת תמונות – משתמשים יכולים להעלות תמונות מהמחשב המקומי.
- אבטחת מידע בסיסית – רק משתמשים מחוברים יכולים לבצע פעולות המשפיעות על המערכת.

8.2 מודלי המערכת

מודל המערכת מתאר את האינטראקציה בין המשתמש לבין המערכת ואת הפעולות המרכזיות שהמערכת מאפשרת לבצע.



המערכת כוללת Actor מרכזי אחד:

- משתמש (User)

המשתמש יכול לבצע את הפעולות הבאות:

- הרשמה למערכת
- התחברות למערכת
- צפייה בפיד
- יצירת פוסט
- העלאת תמונה
- ביצוע Like ו-Unlike
- צפייה בפרופיל אישי
- התנתקות מהמערכת

תרשים Use Case מציג את הקשר בין המשתמש לבין הפעולות המרכזיות במערכת.

8.3 אפיון פונקציונלי וביצועים עיקריים

המערכת כוללת מספר פונקציות מרכזיות:

- הרשמת משתמשים חדשים
- התחברות למערכת
- צפייה בפיד אישי
- יצירת פוסטים
- העלאת תמונות
- ביצוע Like ו-Unlike
- צפייה בפרופיל אישי
- שמירת Preference Vector
- REST API לשליפת נתונים

המערכת מחשבת פיד מותאם אישית באמצעות מנגנון Content-Based Filtering (CBF).
לכל משתמש נשמר Preference Vector המכיל תגיות ומשקלים המייצגים את תחומי העניין שלו.

לכל פוסט מחושב Score לפי הנוסחה:

$$\text{Score} = \sum (\text{weight of matching tags})$$

פוסטים בעלי Score גבוה יותר יוצגו גבוה יותר בפיד.

המערכת נדרשה לספק:

- טעינה מהירה של הפיד
- שמירת נתונים יציבה
- תגובה מהירה לפעולות משתמש
- הצגת תמונות בצורה תקינה
- עדכון מידע בזמן אמת לאחר Like ו-Unlike

8.4 אילוצי המערכת

במהלך פיתוח המערכת היו מספר אילוצים טכנולוגיים ותפעוליים

האילוצים המרכזיים היו:

- פיתוח המערכת במסגרת זמן מוגבלת
 - שימוש בטכנולוגיות שנלמדו במסגרת הלימודים
 - עבודה עם מספר משתמשים קטן יחסית
 - אחסון תמונות מקומי במקום שירות ענן
 - פיתוח המערכת בסביבה מקומית בלבד
 - היעדר שימוש במודלי Machine Learning מורכבים
- בנוסף, מנגנון ההמלצות נדרש להיות פשוט יחסית למימוש אך עדיין להציג התאמה אישית יעילה למשתמש.
- למרות האילוצים, המערכת הצליחה לספק את כל הפונקציות המרכזיות שתוכננו בתחילת הפרויקט.

9. תיאור הארכיטקטורה של המערכת

9.1 ארכיטקטורת המערכת

המערכת פותחה בארכיטקטורת 3 שכבות (Three Layer Architecture), המאפשרת הפרדה בין ממשק המשתמש, הלוגיקה העסקית ומסד הנתונים.

השכבה הראשונה היא שכבת ה-Frontend. שכבה זו אחראית על ממשק המשתמש ומאפשרת אינטראקציה עם המערכת דרך דפדפן האינטרנט. השכבה פותחה באמצעות Vaadin וכוללת את מסכי המערכת כגון HomeView, FeedView, LoginView, RegisterView, ProfileView ו-CreatePostView.

השכבה השנייה היא שכבת ה-Backend. שכבה זו פותחה באמצעות Spring Boot ואחראית על הלוגיקה העסקית של המערכת. בשכבה זו נמצאים שירותי המערכת כגון:

- UserService
- PostService
- FeedService
- RecommendationService

שירותים אלו אחראים על ניהול המשתמשים, ניהול הפוסטים, יצירת הפיד המותאם אישית וחישוב ציוני ההתאמה.

השכבה השלישית היא שכבת מסד הנתונים. המערכת משתמשת ב-MongoDB Atlas כמסד נתונים בענן לצורך שמירת משתמשים, פוסטים ונתוני מערכת נוספים.

זרימת המידע במערכת מתבצעת בצורה הבאה:

המשתמש מבצע פעולה דרך ממשק המשתמש → הבקשה נשלחת לשרת Spring Boot → הלוגיקה העסקית מעבדת את הנתונים → מתבצעת גישה למסד הנתונים → התוצאה מוחזרת למשתמש.

תרשים ארכיטקטורת המערכת

Gallery Miniatures

Three Layer Architecture Model

Layer 1 — Presentation / Client Layer

What the user sees and interacts with

Web Browser Google Chrome Vaadin UI Views HomeView LoginView RegisterView FeedView
CreatePostView ProfileView MainMenuView NavBar

HTTP / HTTPS

Layer 2 — Application / Business Logic Layer

Server side processing and system logic

Spring Boot Server Embedded Tomcat Vaadin Framework REST Controllers UserController
PostController Services UserService PostService FeedService RecommendationService
FileStorageService SessionManager CBF Algorithm

Repository Calls / MongoDB Driver

Layer 3 — Data Layer

Data access, storage, and uploaded files

Repositories UserRepository PostRepository MongoDB Java Driver MongoDB Atlas Collections: users, posts
Local Folder: uploads/

The separation into layers makes the system easier to maintain, test, and extend in the future.

9.2 מודל שרת-לקוח (Client-Server Model)

המערכת פועלת במודל Client-Server.

במודל זה, צד הלקוח (Client) הוא דפדפן האינטרנט של המשתמש, המציג את ממשק המשתמש ומאפשר אינטראקציה עם המערכת.

צד השרת (Server) מבוסס על Spring Boot ומבצע את עיבוד הבקשות, הלוגיקה העסקית, ניהול המשתמשים, יצירת הפיד וניהול הנתונים.

הלקוח שולח בקשות לשרת באמצעות HTTP, והשרת מחזיר תשובות בהתאם לפעולות שהתבקשו. מודל זה מאפשר הפרדה ברורה בין ממשק המשתמש לבין הלוגיקה העסקית של המערכת.

9.3 תהליכים של מערכת ההפעלה

המערכת פועלת בסביבת Web ומבוססת על תקשורת רשת באמצעות פרוטוקול HTTP.

שרת Spring Boot פועל על פורט 8080 ומאזין לבקשות המגיעות מדפדפן המשתמש.

המערכת משתמשת בכרטיס הרשת של המחשב לצורך שליחת וקבלת מידע דרך האינטרנט.

בעת התחברות למערכת או ביצוע פעולה כגון Like, Create Post או Login, נשלחת בקשה מהלקוח אל השרת, והשרת מחזיר תגובה מתאימה לאחר עיבוד הנתונים.

בנוסף, המערכת מבצעת תקשורת מול MongoDB Atlas בענן לצורך שמירת הנתונים ושליפתם.

9.4 פרוטוקולי תקשורת

המערכת משתמשת במספר פרוטוקולי תקשורת מרכזיים:

- HTTP – פרוטוקול התקשורת הראשי בין דפדפן המשתמש לבין שרת Spring Boot. הפרוטוקול משמש לשליחת בקשות ולקבלת תגובות מהשרת.
- HTTPS – גרסה מאובטחת של HTTP המשמשת להצפנת המידע המועבר בין הלקוח לשרת.
- TCP/IP – פרוטוקול התקשורת הבסיסי של רשת האינטרנט, האחראי על העברת המידע בין המחשבים ברשת.
- MongoDB Protocol – פרוטוקול פנימי המשמש לתקשורת בין שרת Spring Boot לבין MongoDB Atlas לצורך שמירה ושליפה של נתונים.

9.5 שירותים חיצוניים ו-REST API

המערכת פועלת בארכיטקטורת רשת מבוססת אינטרנט.

המשתמשים מתחברים למערכת דרך דפדפן אינטרנט, והבקשות מועברות לשרת האפליקציה באמצעות רשת האינטרנט.

שרת Spring Boot מתקשר עם MongoDB Atlas בענן לצורך גישה למסד הנתונים.

מבנה זה מאפשר גישה למערכת מכל מחשב בעל חיבור לאינטרנט.

9.6 סיכום הארכיטקטורה

המערכת עושה שימוש במספר שירותים חיצוניים:

- MongoDB Atlas – שירות מסד נתונים בענן המשמש לשמירת נתוני המשתמשים והפוסטים.
- REST API – המערכת מבוססת על REST לצורך תקשורת בין רכיבי המערכת.
- Vaadin Framework – מסגרת פיתוח לממשק המשתמש של המערכת.
- Spring Boot – Framework לפיתוח צד השרת והלוגיקה העסקית.

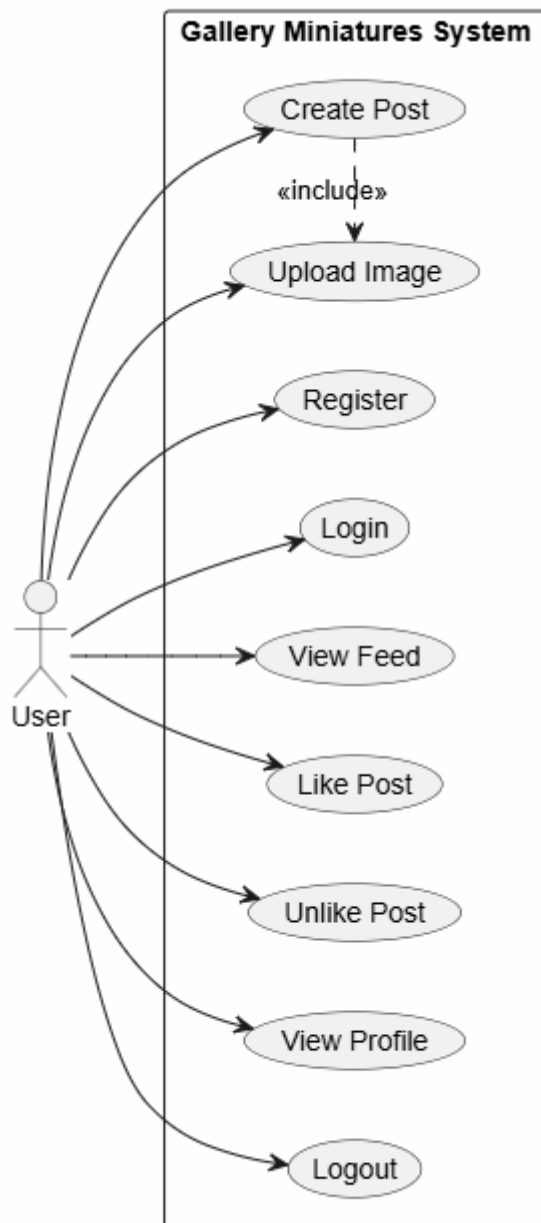
10. ניתוח תרחישים וזרימת מידע

פרק זה מתאר את אופן פעולת המערכת באמצעות תרחישי שימוש (Use Cases) וניתוח זרימת המידע בין המשתמש, ממשק המשתמש, שכבת השירותים ומסד הנתונים. מטרת הפרק היא להמחיש כיצד הדרישות הפונקציונליות שהוגדרו בפרקי האפיון ממומשות בפועל במערכת.

המערכת כוללת מספר תרחישי שימוש מרכזיים:

- רישום משתמש חדש
- התחברות למערכת
- צפייה בפיד מותאם אישית
- ביצוע Like ו-Unlike
- יצירת פוסט חדש
- העלאת תמונה
- צפייה בפרופיל אישי
- קבלת המלצות מותאמות אישית
- שימוש ב-REST API

10.1 תרחיש שימוש – רישום משתמש חדש



מהלך התרחיש

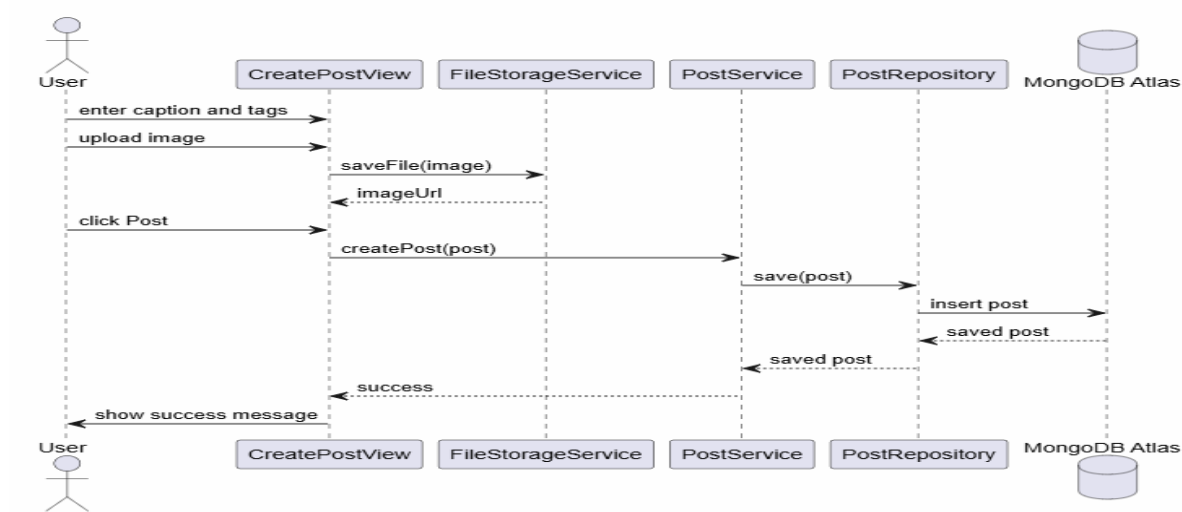
1. המשתמש פותח את מסך ההרשמה.
2. המשתמש מזין שם משתמש, סיסמה ודוא"ל.
3. המשתמש מזין תחומי עניין ראשוניים.
4. המשתמש לוחץ על כפתור Register.
5. RegisterView יוצר אובייקט User.
6. UserService.register() בודק תקינות נתונים.
7. UserRepository.save() שומר את המשתמש.
8. הנתונים נכתבים ל-users.
9. מוצגת הודעת הצלחה.
10. המשתמש מועבר למסך ההתחברות.

תוצאה

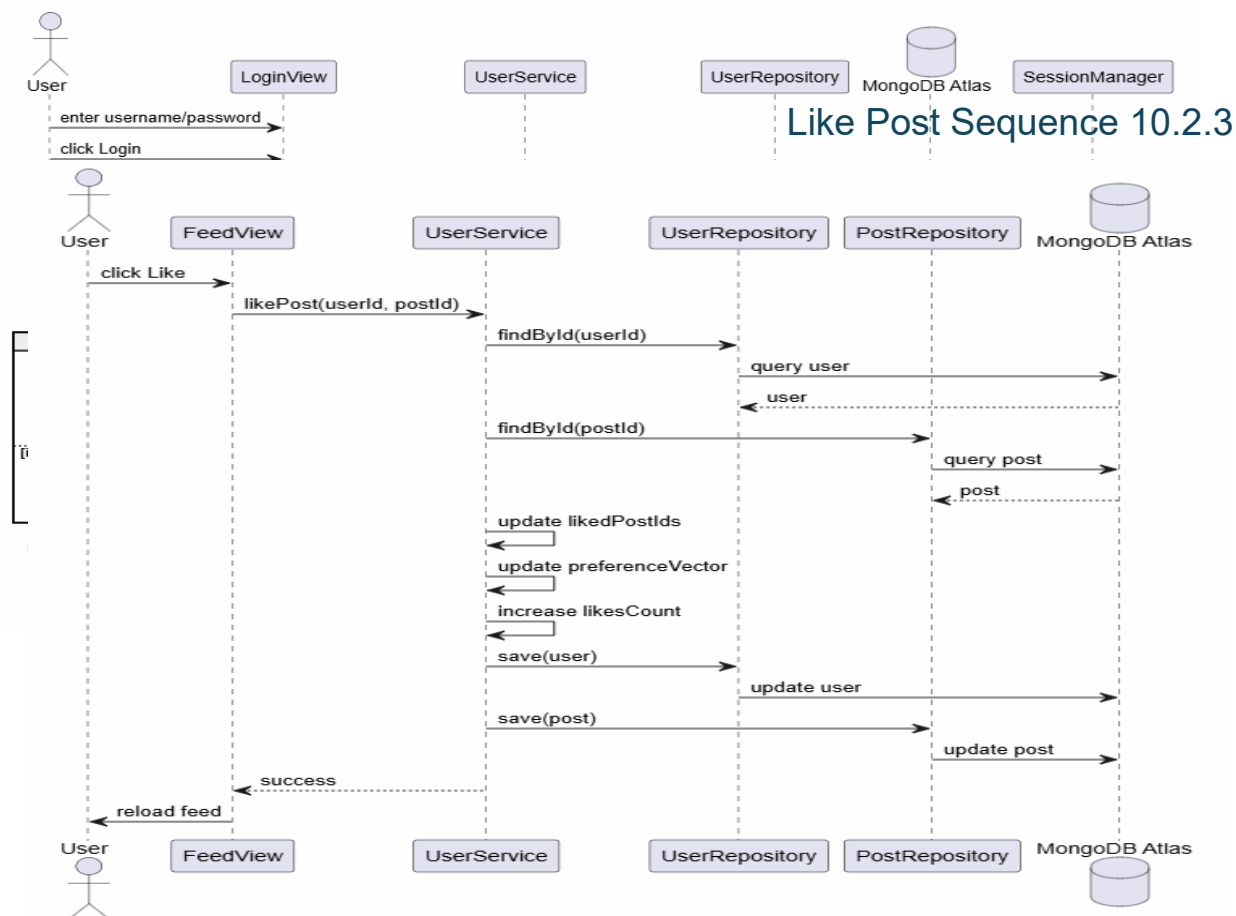
נוצר משתמש חדש במסד הנתונים וניתן להתחבר באמצעות פרטיו.

Sequence Diagram תרשימי רצף 10.2

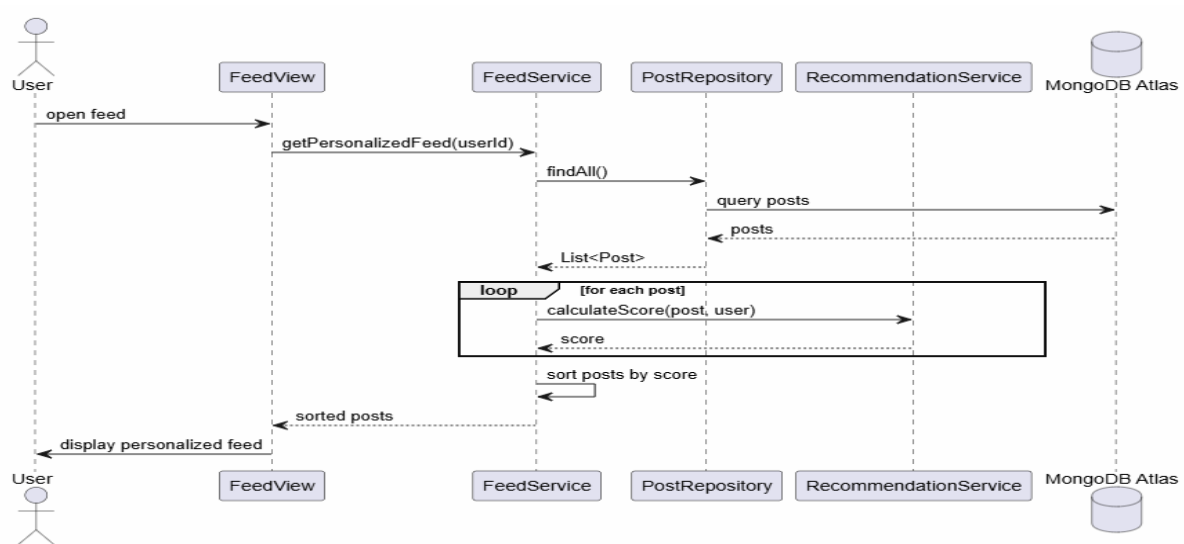
Login Sequence 10.2.1



Create Post Sequence 10.2.2



Personalized Feed Sequence 10.2.4



10.3 תרחיש שימוש – צפייה בפייד מותאם אישית

מטרה

להציג למשתמש פייד תוכן המותאם להעדפותיו.

שחקנים

- משתמש מחובר
- FeedView
- FeedService
- RecommendationService
- MongoDB Atlas

תנאים מקדימים

- המשתמש מחובר למערכת.
- קיימים פוסטים במסד הנתונים.

מהלך התרחיש

1. המשתמש נכנס ל-FeedView.
2. FeedView מקבל את המשתמש המחובר מ-SessionManager.
3. FeedService.getPersonalizedFeed() מופעל.
4. PostRepository שולף את כל הפוסטים.
5. עבור כל פוסט :
 - RecommendationService.calculateScore() מחשב ציון התאמה.
6. הפוסטים ממוינים לפי הציון.
7. הפיד מוצג למשתמש.

תוצאה

המשתמש רואה פוסטים הרלוונטיים להעדפותיו בראש הפיד.

10.4 תרחיש שימוש – ביצוע Like

מטרה

לאפשר למשתמש לסמן פוסט כאהוב ולעדכן את מנגנון ההמלצות.

שחקנים

- משתמש מחובר
- FeedView
- UserService
- MongoDB Atlas

תנאים מקדימים

- המשתמש מחובר.
- הפוסט קיים.
- המשתמש עדיין לא ביצע Like לפוסט.

מהלך התרחיש

1. המשתמש לוחץ על כפתור Like.
2. `FeedView` קורא ל `-UserService.likePost()`.
3. מזהה הפוסט נוסף ל `-likedPostIds`.
4. `likesCount` של הפוסט גדל ב-1.
5. תגיות הפוסט מעדכנות את `preferenceVector`.
6. הנתונים נשמרים ב `-MongoDB Atlas`.
7. הפיד נטען מחדש.

תוצאה

הפוסט מסומן כאהוב והעדפות המשתמש מתעדכנות.

10.5 תרחיש שימוש – הסרת Like

מטרה

לאפשר למשתמש להסיר Like מפוסט ולעדכן את וקטור ההעדפות.

תנאים מקדימים

- המשתמש כבר ביצע Like לפוסט.

מהלך התרחיש

1. המשתמש לוחץ על `Remove Like`.
2. `FeedView` קורא ל `-UserService.unlikePost()`.
3. מזהה הפוסט מוסר מ `-likedPostIds`.
4. `likesCount` קטן ב-1.
5. משקלי התגיות מופחתים ב `-preferenceVector`.
6. הנתונים נשמרים במסד הנתונים.

7. הפיד מתעדכן מחדש .

תוצאה

הלייק מוסר והעדפות המשתמש משתנות בהתאם.

10.6 תרחיש שימוש – יצירת פוסט חדש

מטרה

לאפשר למשתמש לפרסם פוסט חדש.

שחקנים

- משתמש מחובר
- CreatePostView
- PostService
- FileStorageService

תנאים מקדימים

- המשתמש מחובר למערכת

.

מהלך התרחיש

1. המשתמש נכנס למסך Create Post.
2. המשתמש מעלה תמונה .
3. המשתמש מזין כיתוב ותגיות .
4. המשתמש לוחץ על Post.
5. FileStorageService שומר את התמונה .
6. CreatePostView יוצר אובייקט Post.
7. PostService.createPost() שומר את הפוסט .
8. הנתונים נכתבים לאוסף posts.
9. המשתמש מועבר לפיד .

תוצאה

פוסט חדש נוצר ומופיע במערכת.

10.7 תרחיש שימוש – צפייה בפרופיל אישי

מטרה

להציג למשתמש את פרטיו האישיים ואת פעילותו במערכת.

מהלך התרחיש

1. המשתמש נכנס ל ProfileView.
2. המערכת שולפת את המשתמש המחובר.
3. מוצגים:
 - שם משתמש
 - דוא"ל
 - תחומי עניין
 - וקטור העדפות
 - פוסטים אישיים
4. PostService שולף את הפוסטים לפי userId.

תוצאה

המשתמש רואה את פרטיו ואת הפוסטים שפרסם.

10.8 תרחיש שימוש – העלאת תמונה

מטרה

לאפשר למשתמש להעלות תמונות מהמחשב המקומי.

מהלך התרחיש

1. המשתמש בוחר קובץ דרך רכיב Upload.
2. הקובץ מועבר לשרת.
3. FileStorageService שומר את הקובץ בתיקיית uploads.
4. WebConfig ממפה את הקובץ ל URL-פנימי.
5. כתובת הקובץ נשמרת בשדה mediaUrl.

תוצאה

התמונה נשמרת ומוצגת בפוסט.

10.9 תרחיש שימוש – שימוש ב REST API-**מטרה**

לאפשר גישה לנתוני המערכת דרך HTTP.

מהלך התרחיש

1. לקוח שולח בקשת GET לשרת.
2. UserController או PostController מקבלים את הבקשה.
3. הבקר קורא לשירות המתאים.
4. הנתונים נשלפים מ MongoDB.
5. Spring Boot ממיר את האובייקטים ל JSON.
6. השרת מחזיר תגובת HTTP.

תוצאה

הלקוח מקבל נתונים בפורמט JSON.

10.10 זרימת המידע במערכת

זרימת המידע במערכת מתבצעת לפי הסדר הבא:

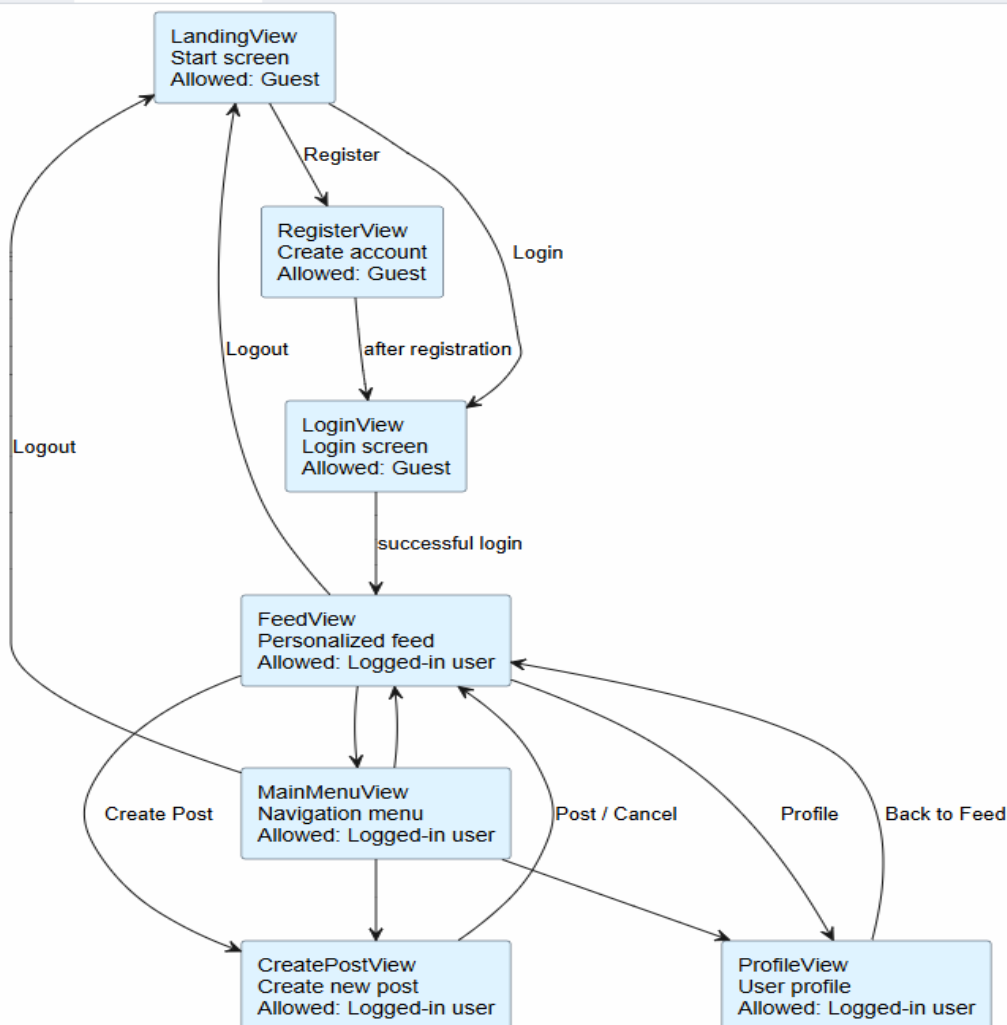
1. המשתמש מבצע פעולה בדפדפן.
2. Vaadin מעביר את האירוע לשרת.
3. השרת מפעיל Service מתאים.
4. ה Service-משתמש ב Repository.
5. ה Repository-מתקשר עם MongoDB Atlas.
6. הנתונים חוזרים לשרת.
7. Vaadin מעדכן את המסך בדפדפן.

10.11 סיכום

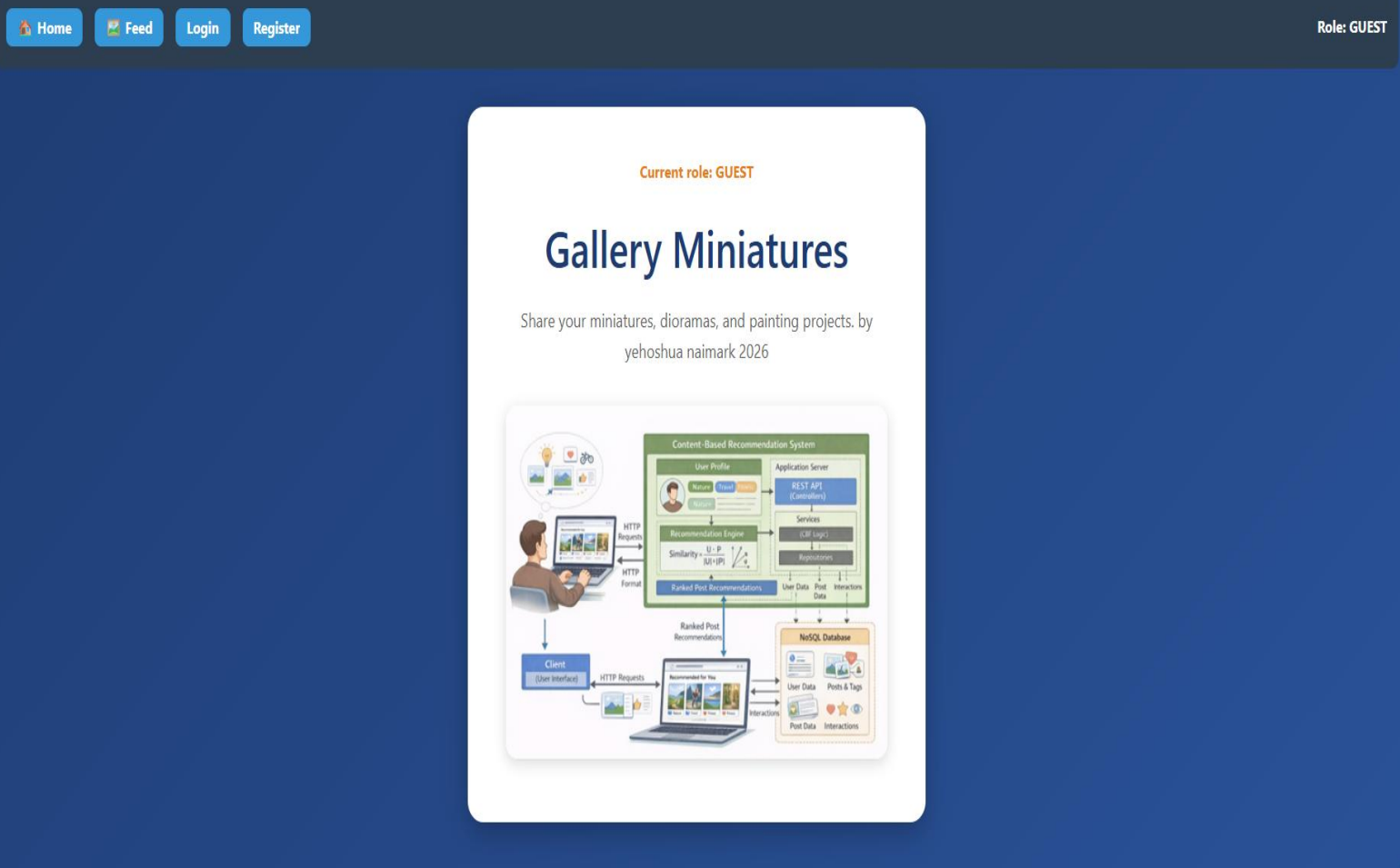
ניתוח התרחישים וזרימת המידע מציג כיצד המשתמש מתקשר עם המערכת וכיצד הנתונים עוברים בין שכבות התוכנה השונות. תרחישים אלו ממחישים את פעולת המערכת בפועל ומדגימים את האינטגרציה בין ממשק המשתמש, הלוגיקה העסקית, מסד הנתונים ומנגנון ההמלצות.

11. תיאור מסכים וממשק משתמש

המערכת כוללת מספר מסכים מרכזיים, שכל אחד מהם ממלא תפקיד שונה בתהליך העבודה של המשתמש.



11.1 מסך פתיחה - LandingView



מסך הבית מהווה את מסך הפתיחה הראשי של מערכת Gallery Miniatures.

מטרת המסך היא להציג את המערכת, לאפשר ניווט ראשוני ולהציג את מצב המשתמש הנוכחי במערכת.

רכיבי ה-UI במסך:

- Navigation Bar – סרגל ניווט עליון המאפשר מעבר בין מסכי המערכת. הסרגל כולל כפתורי Home, Feed, Login ו-Register. כאשר המשתמש מחובר מופיעים גם Create Post, Profile ו-Logout.
- Paragraph – מציג את תפקיד המשתמש הנוכחי במערכת (Role), לדוגמה: GUEST או USER.
- H1 – כותרת ראשית המציגה את שם המערכת "Gallery Miniatures".

- Paragraph – תיאור קצר של מטרת המערכת ושיתוף פרויקטי מיניאטורות ודיורמות.
- Image – תמונה מרכזית המציגה את רעיון מערכת ההמלצות ומערכת שיתוף הפוסטים.

• Button – כפתורי ניווט:

- Feed – מעבר לפיד הפוסטים.

- Login – מעבר למסך התחברות.

- Register – מעבר למסך הרשמה.

המסך זמין גם למשתמשי Guest וגם למשתמשים מחוברים.

11.2 תיאור מסכים

RegisterView 11.2.1

Username

מסך ההרשמה מאפשר יצירת משתמש חדש במערכת.

רכיבי ה-UI במסך:

Email

• TextField – Username

מאפשר למשתמש להזין שם משתמש.

• EmailField – Email

מאפשר הזנת כתובת דואר אלקטרוני.

Password

• PasswordField – Password

מאפשר הזנת סיסמה בצורה מוסתרת.

Register

• Button – Register

שולח את נתוני המשתמש לשרת לצורך יצירת חשבון חדש.

לאחר הרשמה מוצלחת המשתמש מועבר למסך ההתחברות.

11.2.2 LoginView

מסך ההתחברות מאפשר למשתמש קיים להתחבר למערכת.

רכיבי ה-UI במסך:

- TextField – Username

מאפשר הזנת שם משתמש.

- PasswordField – Password

מאפשר הזנת סיסמה בצורה מאובטחת.

- Button – Login

מאמת את פרטי המשתמש מול מסד הנתונים.

במקרה של התחברות מוצלחת המשתמש מועבר לפיד האישי שלו.

במקרה של שגיאה מוצגת הודעת Notification מתאימה.

11.2.3. מסך הפיד -FeedView

מסך הפיד מציג את הפוסטים במערכת.

Miniatures & Dioramas Feed

כאשר המשתמש מחובר, הפיד מוצג לפי מנגנון ההמלצות של המערכת. כאשר המשתמש הוא Guest, מוצג פיד ציבורי רגיל.

Feed Create Post Profile Logout

רכיבי ה-UI במסך:

- Navigation Bar – סרגל ניווט עליון למעבר בין מסכי המערכת.
- H1 – כותרת המסך "Miniatures & Dioramas Feed".
- VerticalLayout – כרטיס פוסט המכיל את נתוני הפוסט.
- H3 – מציג את כותרת או תיאור הפוסט.
- Image – מציג את תמונת הפוסט.
- Paragraph – מציג את תגיות הפוסט.
- Paragraph – מציג את מספר הלייקים של הפוסט.
- Button – Like / Remove Like
- מאפשר לבצע Like או Unlike לפוסט.
- Button – Login to Like
- מופיע עבור משתמש Guest ומעביר למסך ההתחברות.
- ביצוע Like מעדכן את Preference Vector של המשתמש ומשפיע על סדר הפוסטים בפיד.

11.2.4. מסך יצירת פוסט -CreatePostView

מסך יצירת הפוסט מאפשר למשתמש מחובר להעלות פוסט חדש למערכת.

רכיבי ה-UI במסך:

• Caption – TextArea

מאפשר הזנת תיאור הפוסט.

• Media Type – TextField

מציג את סוג המדיה של הפוסט.

• Tags – TextField

Create New Post

Caption

Describe your miniature, diorama, or painting project...

Media Type

image

Tags (comma separated)

warhammer40k, painting, blue, gold, weathering

Upload File...

Drop file here

Post

Cancel

מאפשר הזנת תגיות מופרדות בפסיקים.

• Upload – רכיב העלאת תמונות מהמחשב המקומי.

• Button – Post

יוצר פוסט חדש ושומר אותו במסד הנתונים.

• Button – Cancel

מבטל את הפעולה וחוזר לפיד.

לאחר יצירת פוסט מוצגת הודעת Notification והמשתמש מועבר לפיד.

User Profile

Username: a

Email: a@gmail.com

Member Since: Mon May 11 13:04:11 IDT 2026

Liked Posts: 1

Interests: No interests recorded yet.

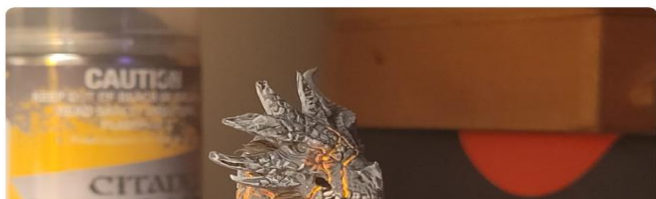
Preference Vector

fantasy: 1

black: 1

dragon: 1

My Posts



11.2.5. מסך פרופיל-ProfileView

מסך הפרופיל מציג את נתוני המשתמש ואת תחומי העניין שלו.

רכיבי ה-UI במסך:

• H1 – כותרת המסך "User Profile".

• Paragraph – מציג את שם המשתמש.

• Paragraph – מציג את כתובת האימייל.

• Paragraph – מציג את תאריך יצירת החשבון.

• Paragraph – מציג את מספר הפוסטים שסומנו ב-Like.

• Paragraph – מציג את תחומי העניין של המשתמש.

• Preference Vector – מציג את התגיות והמשקלים שנשמרו עבור המשתמש.

• My Posts – רשימת הפוסטים שפורסמו על ידי המשתמש.

המסך מאפשר למשתמש לראות כיצד מנגנון ההמלצות לומד את תחומי העניין שלו.

11.3 עיצוב הממשק

המערכת משתמשת בעיצוב מודרני המבוסס על:

- Cards
- Rounded Corners
- Shadows
- צבעים דינמיים
- Navigation Bar

- Responsive Layouts

בנוסף נעשה שימוש ב:

- VerticalLayout

- HorizontalLayout

- Button

- Image

- Paragraph

- Notification

11.4 ניווט בין מסכים

הניווט מתבצע באמצעות `@Route` של Vaadin.

דוגמאות:

```
@Route("feed")
```

```
@Route("profile")
```

```
@Route("create-post")
```

מעבר בין מסכים מתבצע באמצעות:

```
ui.navigate("feed");
```

11.10 הגנת מסכים

מסכים פרטיים מוגנים באמצעות `SessionManager`.

אם משתמש אינו מחובר:

- הוא מועבר למסך `Login`.

- נמנעת גישה לפיד ולפרופיל.

11.11 סיכום

ממשק המשתמש שנבנה בפרויקט מספק חוויית שימוש מודרנית, ברורה ואינטראקטיבית. המסכים השונים מאפשרים למשתמש לבצע את כל הפעולות המרכזיות במערכת – החל מהרשמה והתחברות ועד יצירת פוסטים וקבלת המלצות מותאמות אישית. השימוש ב-Vaadin אפשר פיתוח

מלא של הממשק בשפת Java תוך אינטגרציה מלאה עם Spring Boot ושכבת השירותים של המערכת.

12. תיאור התוכנה

פרק זה מתאר את מבנה התוכנה שפותחה במסגרת הפרויקט, את סביבת הפיתוח, את מבנה המחלקות, את מבני הנתונים המרכזיים ואת הפעולות והאלגוריתמים החשובים במערכת. המערכת פותחה בשפת Java תוך שימוש במסגרת Spring Boot בצד השרת, Vaadin לבניית ממשק המשתמש ו-MongoDB Atlas כמסד נתונים בענן. התוכנה בנויה בארכיטקטורת שכבות, המפרידה בין ממשק המשתמש, הלוגיקה העסקית, שכבת הגישה לנתונים ומסד הנתונים.

12.1 סביבת הפיתוח של התוכנה

סביבת הפיתוח של המערכת נבחרה בהתאם לצורכי הפרויקט ולדרישות הפיתוח של מערכת Web מודרנית. סביבת הפיתוח כללה מערכת הפעלה, סביבת כתיבת קוד, מסגרות פיתוח, שפות תכנות וספריות עזר.

12.1.1 מערכת הפעלה

המערכת פותחה על מערכת ההפעלה Windows 10/11. מערכת הפעלה זו שימשה להרצת סביבת הפיתוח, שרת Spring Boot המקומי, הדפדפן וכלי הבדיקה.

12.1.2 סביבות הפיתוח (IDEs)

סביבת הפיתוח המרכזית הייתה Visual Studio Code. סביבה זו שימשה לכתיבת הקוד, ניהול קבצי הפרויקט, הרצת השרת ובדיקת שגיאות קומפילציה. בנוסף נעשה שימוש ב-Terminal להרצת פקודות Maven ולהפעלת היישום.

12.1.3 מסגרות לפיתוח התוכנה (Frameworks)

בפרויקט נעשה שימוש במספר Frameworks מרכזיים:

- Spring Boot – שימש לפיתוח צד השרת, ניהול רכיבי המערכת, יצירת Services, Repositories ו-REST Controllers.
- Vaadin – שימש לפיתוח ממשק המשתמש הגרפי של המערכת באמצעות Java.
- Spring Data MongoDB – שימש לחיבור בין המערכת לבין MongoDB Atlas ולביצוע פעולות מול מסד הנתונים.

12.1.4 שפות תכנות

שפת התכנות המרכזית בפרויקט היא Java.

Java נבחרה מכיוון שהיא שפה מונחית עצמים, יציבה, מתאימה לפיתוח מערכות שרת, ומשתלבת בצורה טובה עם Spring Boot ו-Vaadin.

12.1.5 ספריות/מערכות/מנגנוני עזר נוספים

במערכת נעשה שימוש במספר ספריות ומנגנוני עזר:

- Maven – לניהול תלויות והרצת הפרויקט.
- MongoDB Atlas – מוד נתונים בענן.
- MongoRepository – לביצוע פעולות CRUD מול MongoDB.
- Vaadin Upload – להעלאת תמונות.
- REST API – לחשיפת נתונים בפורמט JSON.
- Postman – לבדיקת נקודות הקצה של ה-API.

12.2 תיאור מבנה הפרויקט והמחלקות

הפרויקט פותח בארכיטקטורת שכבות (Layered Architecture) במטרה ליצור מערכת מודולרית, מסודרת ונוחה לתחזוקה ולהרחבה עתידית.

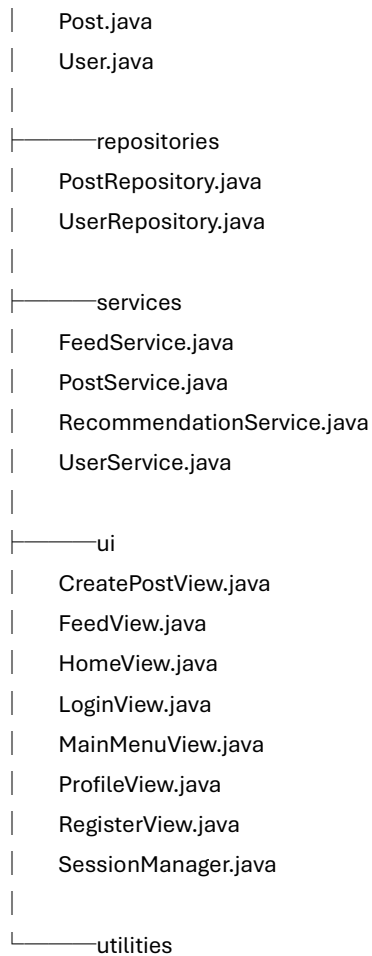
מבנה הפרויקט מחולק למספר חבילות (Packages), כאשר כל חבילה אחראית על תחום שונה במערכת. הפרדה זו מאפשרת חלוקת אחריות ברורה בין רכיבי המערכת ומונעת תלות ישירה בין שכבות שונות.

12.2.1 עץ המודולים של הפרויקט

```

├── finalp
│   ├── AppMain.java
│   └──
│       ├── config
│       │   └── WebConfig.java
│       └──
│           ├── controllers
│           │   ├── AuthController.java
│           │   ├── FeedController.java
│           │   ├── PostController.java
│           │   └── UserController.java
│           └──
│               ├── datamodels
│               └── Interaction.java

```



המערכת מחולקת למספר חבילות עיקריות:

• datamodels

מכילה את מחלקות הנתונים של המערכת, כגון User ו-Post.

• repositories

מכילה את מחלקות הגישה למסד הנתונים באמצעות MongoRepository.

• services

מכילה את הלוגיקה העסקית של המערכת, כולל מנגנון ההמלצות, ניהול משתמשים וניהול פוסטים.

• controllers

מכילה את מחלקות ה-REST API.

• ui

מכילה את מסכי Vaadin וממשק המשתמש של המערכת.

• config

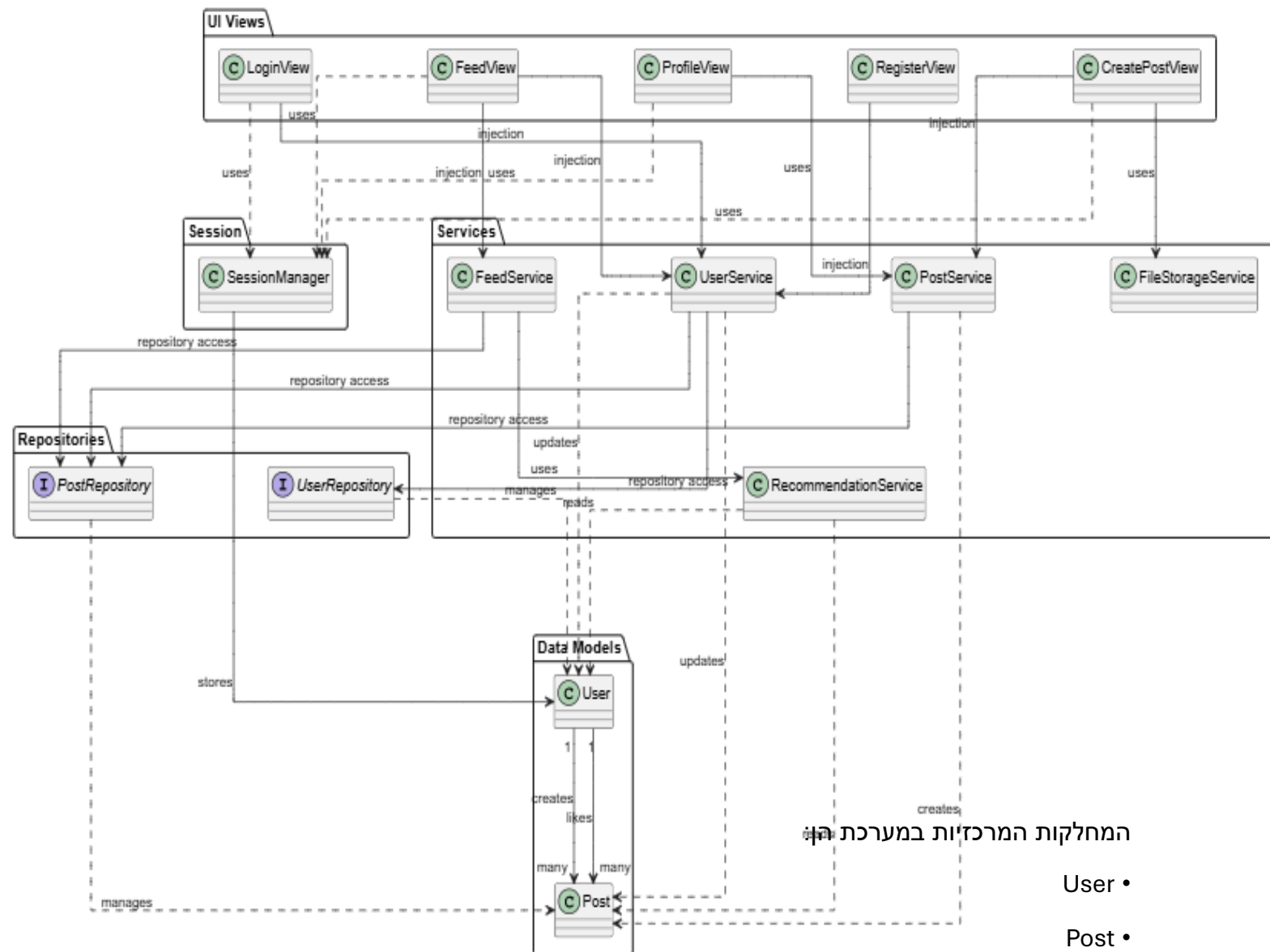
מכילה קובצי קונפיגורציה כגון WebConfig.

מבנה זה מאפשר ארגון נכון של הקוד והפרדה בין ממשק המשתמש, הלוגיקה העסקית ומסד הנתונים.

12.2.2 תרשים המחלקות UML Class Diagram

תרשים המחלקות (UML Class Diagram) מציג את המחלקות המרכזיות במערכת ואת הקשרים ביניהן.

Gallery Miniatures - Class Relationships



המחלקות המרכזיות במערכת הן:

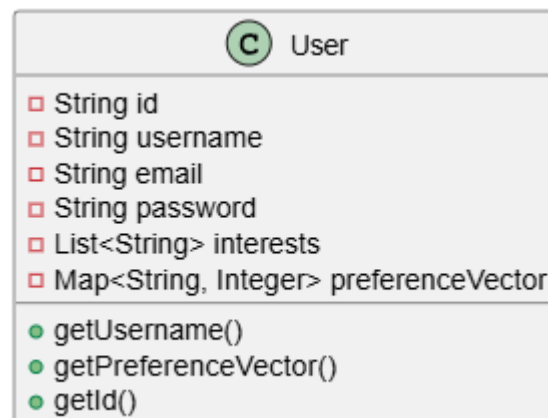
- User •
- Post •
- UserService •
- PostService •
- FeedService •
- RecommendationService •
- UserRepository •
- PostRepository •

- FeedView
- CreatePostView
- ProfileView
- SessionManager

התרשים מציג את מבנה המחלקות, התכונות המרכזיות והקשרים בין רכיבי המערכת.

12.2.3 תיאור מחלקות מפורט

מחלקת User מייצגת משתמש במערכת.

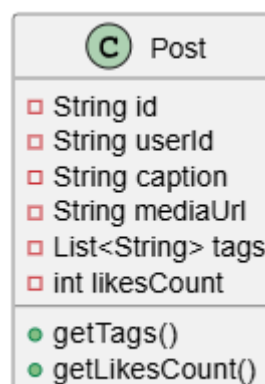


המחלקה כוללת מידע כגון:

- שם משתמש
- כתובת אימייל
- סיסמה
- תחומי עניין
- Preference Vector
- רשימת פוסטים שסומנו ב-Like

מחלקה זו נשמרת במסד הנתונים באוסף users.

מחלקת Post מייצגת פוסט במערכת.



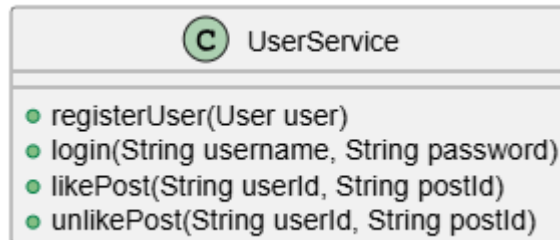
המחלקה כוללת:

- מזהה פוסט
- מזהה יוצר הפוסט
- תיאור הפוסט
- כתובת תמונה
- תגיות
- מספר לייקים
- תאריך יצירה

מחלקה זו נשמרת באוסף posts במסד הנתונים.

מחלקת UserService אחראית על ניהול המשתמשים במערכת.

המחלקה מבצעת פעולות כגון:



- הרשמת משתמשים
- התחברות למערכת
- Like ו-Unlike
- עדכון Preference Vector
- שמירת נתוני משתמשים

מחלקת PostService אחראית על ניהול הפוסטים במערכת.



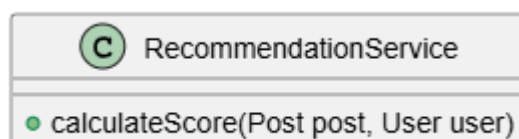
- המחלקה מבצעת:
- יצירת פוסטים
 - שמירת פוסטים
 - שליפת פוסטים
 - ניהול מידע הקשור לתוכן המוצג במערכת

מחלקת RecommendationService מממשת את מנגנון ההמלצות של המערכת.

המחלקה מחשבת ציון התאמה (Score) לכל פוסט על פי תגיות הפוסט והעדפות המשתמש.

החישוב מבוסס על Content-Based Filtering (CBF).

מחלקת FeedService אחראית על יצירת הפיד האישי של המשתמש.



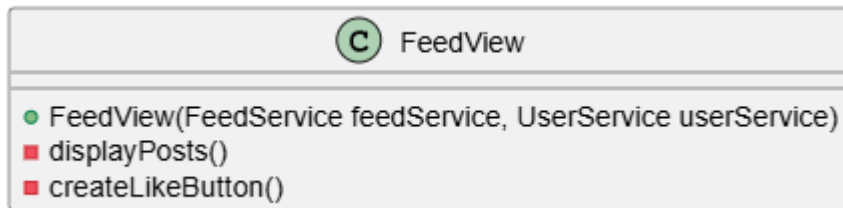
- המחלקה:
- שולפת פוסטים ממסד הנתונים
 - מחשבת ציון התאמה לכל פוסט
 - ממיינת את הפוסטים לפי רלוונטיות

מחלקות ה-UI אחראיות על ממשק המשתמש במערכת.

המחלקות המרכזיות הן:

- מחלקת FeedView אחראית על הצגת פיד הפוסטים במערכת.
- המחלקה מקבלת דרך הבנאי את FeedService ואת UserService.

FeedService משמש לשליפת הפוסטים ולהצגת הפיד המתאים למשתמש, ו-UService משמש לביצוע פעולות Like ו-Unlike.



המחלקה מבצעת פעולות כגון:

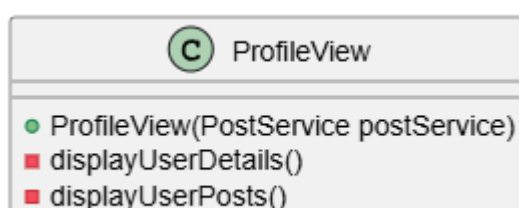
- הצגת רשימת הפוסטים במערכת
- הצגת תמונות, תגיות ומספר לייקים לכל פוסט
- בדיקה האם המשתמש מחובר או Guest
- הצגת פיד ציבורי למשתמש Guest
- הצגת פיד מותאם אישית למשתמש מחובר
- יצירת כפתור Like או Remove Like
- עדכון הפיד לאחר ביצוע Like או Unlike

הפעולה displayPosts() אחראית על הצגת הפוסטים במסך.

הפעולה createLikeButton() אחראית על יצירת כפתור Like או Remove Like בהתאם למצב הפוסט אצל המשתמש.

- מחלקת ProfileView אחראית על הצגת הפרופיל האישי של המשתמש המחובר.

המחלקה מקבלת דרך הבנאי את PostService, ומשתמשת בו לצורך הצגת הפוסטים שנוצרו על ידי המשתמש.



המחלקה מבצעת פעולות כגון:

- הצגת שם המשתמש

- הצגת כתובת האימייל של המשתמש
- הצגת תאריך יצירת החשבון
- הצגת מספר הפוסטים שסומנו ב-Like
- הצגת תחומי העניין של המשתמש
- הצגת Preference Vector
- הצגת הפוסטים שהמשתמש פרסם

הפעולה `displayUserDetails()` אחראית על הצגת פרטי המשתמש.

הפעולה `displayUserPosts()` אחראית על הצגת הפוסטים שנוצרו על ידי המשתמש.

- מחלקת `CreatePostView` אחראית על יצירת פוסט חדש במערכת.

המחלקה מקבלת דרך הבנאי את `PostService` ואת `FileStorageService`.

`PostService` משמש לשמירת הפוסט במסד הנתונים, ו-`FileStorageService` משמש לשמירת התמונה שהמשתמש מעלה לתיקיית `uploads`.

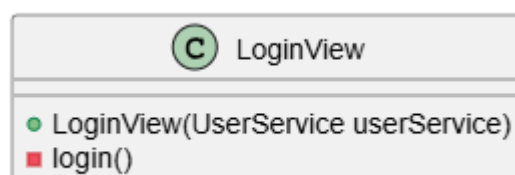


המחלקה מבצעת פעולות כגון:

- בדיקה שהמשתמש מחובר למערכת
- הצגת טופס יצירת פוסט
- קבלת Caption מהמשתמש

- קבלת Tags מופרדים בפסיקים
- קבלת סוג המדיה של הפוסט
- העלאת תמונה מהמחשב המקומי
- שמירת התמונה באמצעות FileStorageService
- שימוש ב-Callback לאחר סיום שמירת התמונה
- יצירת אובייקט Post חדש
- שליחת הפוסט ל-PostService לצורך שמירה במסד הנתונים
- הצגת הודעות שגיאה או הצלחה למשתמש
- הפעולה createPost() אחראית על יצירת אובייקט Post מתוך הנתונים שהוזנו בטופס ושליחתו לשכבת השירותים.
- הפעולה uploadImage() אחראית על קבלת קובץ התמונה מהמשתמש ושמירתו. לאחר שהשמירה מסתיימת, מופעל Callback מסוג FileUploadCallback שמעדכן את נתיב התמונה או מציג הודעת שגיאה במקרה של כשל.

- מחלקת LoginView אחראית על התחברות משתמשים קיימים למערכת.
- המחלקה מקבלת דרך הבנאי את UserService, ומשתמשת בו לצורך בדיקת פרטי ההתחברות מול מסד הנתונים.



- המחלקה מבצעת פעולות כגון:
- הצגת שדה להזנת שם משתמש
- הצגת שדה להזנת סיסמה

- בדיקת פרטי ההתחברות
- שמירת המשתמש המחובר ב-SessionManager
- מעבר למסך הפיד לאחר התחברות מוצלחת
- הצגת הודעת שגיאה במקרה של פרטי התחברות שגויים
- הפעולה login() אחראית על קבלת שם המשתמש והסיסמה, שליחתם ל-UserService, ובמקרה של הצלחה שמירת המשתמש הנוכחי במערכת.
- מחלקת RegisterView אחראית על רישום משתמש חדש למערכת.
- המחלקה מקבלת דרך הבנאי את UserService, ומשתמשת בו לצורך יצירת משתמש חדש ושמירתו במסד הנתונים.



- המחלקה מבצעת פעולות כגון:
- הצגת שדה להזנת שם משתמש
- הצגת שדה להזנת אימייל
- הצגת שדה להזנת סיסמה
- יצירת אובייקט User חדש
- שליחת המשתמש החדש ל-UserService
- שמירת המשתמש במסד הנתונים
- מעבר למסך ההתחברות לאחר הרשמה מוצלחת
- הפעולה registerUser() אחראית על קבלת פרטי המשתמש מהטופס, יצירת אובייקט User ושליחתו לשכבת השירותים לצורך שמירה.
- מחלקות אלו נבנו באמצעות Vaadin ומאפשרות למשתמש לבצע את כל פעולות המערכת דרך ממשק גרפי.

12.3 מבני נתונים בהם נעשה שימוש

במהלך פיתוח המערכת נעשה שימוש במספר מבני נתונים מרכזיים לצורך שמירת מידע, ניהול פוסטים וחישוב מנגנון ההמלצות.

• <List<String

שימש לשמירת תגיות (Tags) של פוסטים ורשימות תחומי עניין של משתמשים.

- `<Map<String, Integer`

שימש למימוש Preference Vector של המשתמש, כאשר כל תגית משויכת למשקל מספרי.

- `ArrayList`

שימש לניהול רשימות פוסטים ומשתמשים בזיכרון.

- `MongoDB Documents`

שימשו לשמירת אובייקטים במסד הנתונים בפורמט BJSON.

מבני נתונים אלו אפשרו עבודה יעילה עם נתוני המשתמשים והפוסטים וחישבו מהיר של ציוני ההתאמה במנגנון ההמלצות.

12.4 קוד התוכנית לפעולות ואלגוריתמים עיקריים וחשובים

`calculateScore()`

טענת כניסה:

הפעולה מקבלת אובייקט מסוג `Post` ואובייקט מסוג `User`.

טענת יציאה:

הפעולה מחזירה מספר שלם (`int`) המייצג את ציון ההתאמה בין הפוסט לבין המשתמש.

תפקיד הפעולה:

הפעולה מחשבת את ציון ההתאמה של פוסט מסוים עבור משתמש מסוים בהתאם לתגיות הפוסט ולהעדפות המשתמש.

```
public int calculateScore(Post post, User user) {
    if (post.getTags() == null ||
        user.getPreferenceVector() == null) {
        return 0;
    }
    int score = 0;
    Map<String, Integer> preferenceVector =
        user.getPreferenceVector();
    for (String tag : post.getTags()) {
        Integer weight = preferenceVector.get(tag);
        if (weight != null) {
            score += weight;
        }
    }
}
```

```

return score;
}

```

הסבר הפעולה:

בתחילת הפעולה מתבצעת בדיקה האם קיימות תגיות לפוסט והאם למשתמש קיים Preference Vector. במקרה שאחד מהם ריק, הפעולה מחזירה 0.

לאחר מכן נוצר משתנה score אשר ישמש לשמירת ציון ההתאמה הכולל.

הפעולה עוברת בלולאה על כל תגיות הפוסט. עבור כל תגית מתבצעת בדיקה האם היא קיימת ב- Preference Vector של המשתמש.

אם נמצאה תגית משותפת, המשקל שלה מתווסף למשתנה score.

בסיום הלולאה מוחזר ציון ההתאמה הכולל.

סיבוכיות זמן ריצה:

$O(n)$

כאשר n הוא מספר התגיות של הפוסט.

likePost()

טענת כניסה:

הפעולה מקבלת מזהה משתמש (userId) ומזהה פוסט (postId).

טענת יציאה:

הפעולה מחזירה אובייקט User מעודכן לאחר ביצוע פעולת Like.

תפקיד הפעולה:

הפעולה אחראית על הוספת Like לפוסט, עדכון מספר הלייקים, שמירת הפוסט ברשימת הפוסטים האהובים של המשתמש ועדכון Preference Vector.

```
public User likePost(String userId, String postId) {  
    User user = userRepository.findById(userId)  
        .orElseThrow();  
    Post post = postRepository.findById(postId)  
        .orElseThrow();  
    if (user.getLikedPostIds() == null) {  
        user.setLikedPostIds(new ArrayList<>());  
    }  
    if (user.getLikedPostIds().contains(postId)) {  
        return user;  
    }  
    user.getLikedPostIds().add(postId);  
    while (user.getLikedPostIds().size() > MAX_RECENT_LIKES) {  
        String removedPostId = user.getLikedPostIds().remove(0);  
        postRepository.findById(removedPostId).ifPresent(oldPost -> {  
            if (oldPost.getLikesCount() > 0) {  
                oldPost.setLikesCount(oldPost.getLikesCount() - 1);  
                postRepository.save(oldPost);  
            }  
        });  
    }  
    post.setLikesCount(post.getLikesCount() + 1);  
    postRepository.save(post);  
    rebuildPreferenceVector(user);  
  
    return userRepository.save(user);  
}
```

הסבר הפעולה

בתחילת הפעולה מתבצעת שליפה של המשתמש והפוסט ממסד הנתונים באמצעות `userRepository` ו-`postRepository`.

אם רשימת הפוסטים האהובים של המשתמש אינה קיימת, נוצרת רשימה חדשה מסוג `ArrayList`. לאחר מכן מתבצעת בדיקה האם המשתמש כבר סימן Like לפוסט. במקרה כזה הפעולה מחזירה את המשתמש ללא שינוי ומונעת כפילות לייקים.

אם הפוסט עדיין לא סומן ב-Like, מזהה הפוסט נוסף לרשימת `likedPostIds` של המשתמש. המערכת מגבילה את מספר הלייקים האחרונים באמצעות `MAX_RECENT_LIKES`. כאשר מספר הלייקים חורג מהמגבלה, הפוסט הישן ביותר מוסר מהרשימה ומספר הלייקים שלו מוקטן. לאחר מכן מספר הלייקים של הפוסט הנוכחי גדל ב-1 והפוסט נשמר במסד הנתונים. לבסוף מתבצע עדכון מחדש של `Preference Vector` באמצעות `rebuildPreferenceVector()`, והמשתמש המעודכן נשמר במסד הנתונים.

סיבוכיות זמן ריצה:

$$O(n)$$

כאשר n הוא מספר הפוסטים ברשימת `likedPostIds`.

`unlikePost()`

טענת כניסה:

הפעולה מקבלת מזהה משתמש (`userId`) ומזהה פוסט (`postId`).

טענת יציאה:

הפעולה מחזירה אובייקט User מעודכן לאחר הסרת ה Like-מהפוסט.

תפקיד הפעולה:

הפעולה אחראית על הסרת Like מפוסט, עדכון מספר הלייקים של הפוסט ועדכון Preference Vector של המשתמש.

```
public User unlikePost(String userId, String postId) {
    User user = userRepository.findById(userId)
        .orElseThrow();

    Post post = postRepository.findById(postId)
        .orElseThrow();

    if (user.getLikedPostIds() == null) {
        user.setLikedPostIds(new ArrayList<>());
    }

    if (user.getLikedPostIds().remove(postId)) {
        if (post.getLikesCount() > 0) {
            post.setLikesCount(post.getLikesCount() - 1);
            postRepository.save(post);
        }

        rebuildPreferenceVector(user);
        return userRepository.save(user);
    }

    return user;
}
```

הסבר הפעולה:

בתחילת הפעולה מתבצעת שליפה של המשתמש והפוסט ממסד הנתונים באמצעות userRepository ו-postRepository.

אם רשימת likedPostIds של המשתמש אינה קיימת, נוצרת רשימה חדשה מסוג ArrayList.

לאחר מכן המערכת מנסה להסיר את מזהה הפוסט מרשימת הפוסטים שסומנו ב-Like. אם ההסרה הצליחה, המערכת מקטינה את מספר הלייקים של הפוסט ושומרת את הפוסט המעודכן במסד הנתונים.

לאחר עדכון מספר הלייקים מתבצע חישוב מחדש של Preference Vector באמצעות rebuildPreferenceVector(), כדי לעדכן את תחומי העניין של המשתמש בהתאם לשינוי.

לבסוף המשתמש המעודכן נשמר במסד הנתונים ומוחזר מהפעולה.

אם הפוסט לא היה קיים ברשימת הלייקים של המשתמש, הפעולה מחזירה את המשתמש ללא שינוי.

סיבוכיות זמן ריצה:

$O(n)$

כאשר n הוא מספר הפוסטים ברשימת `likedPostIds`.

`getPersonalizedFeed()`

טענת כניסה:

הפעולה מקבלת מזהה משתמש `(userId)`.

טענת יציאה:

הפעולה מחזירה רשימה מסוג `List<Post>` המכילה את הפוסטים ממוינים לפי רמת התאמה למשתמש.

תפקיד הפעולה:

הפעולה אחראית על יצירת הפיד האישי של המשתמש באמצעות מנגנון ההמלצות של המערכת.

```
public List<Post> getPersonalizedFeed(String userId) {
    User user = userRepository.findById(userId)
        .orElseThrow();

    List<Post> posts = postRepository.findAll();
    return posts.stream()
        .sorted((p1, p2) ->
            Integer.compare(
                recommendationService.calculateScore(p2, user),
                recommendationService.calculateScore(p1, user)
            ))
        .toList();
}
```

הסבר הפעולה:

בתחילת הפעולה מתבצעת שליפה של המשתמש ממסד הנתונים באמצעות `userRepository`.

לאחר מכן המערכת שולפת את כל הפוסטים ממסד הנתונים באמצעות `postRepository`.

המערכת משתמשת ב-Stream API של Java לצורך מיון הפוסטים. עבור כל פוסט מופעלת הפעולה `calculateScore()` מתוך `RecommendationService`, אשר מחשבת את רמת ההתאמה בין הפוסט לבין המשתמש.

הפוסטים ממוינים בסדר יורד לפי Score, כך שפוסטים בעלי ציון התאמה גבוה יותר יוצגו ראשונים בפייד.

לבסוף מוחזרת רשימת פוסטים ממוינת המייצגת את הפיד האישי של המשתמש.

סיבוכיות זמן ריצה:

$$O(n \log n)$$

כאשר n הוא מספר הפוסטים במערכת.

`createPost()`

טענת כניסה:

הפעולה מקבלת אובייקט מסוג `Post`.

טענת יציאה:

הפעולה מחזירה אובייקט `Post` לאחר שמירתו במסד הנתונים.

תפקיד הפעולה:

הפעולה אחראית על יצירת פוסט חדש במערכת, אתחול נתוני ברירת מחדל ושמירת הפוסט במסד הנתונים.

```
public Post createPost(Post post) {
    post.setCreatedAt(new Date());
    post.setLikesCount(0);
    return postRepository.save(post);
}
```

הסבר הפעולה:

בתחילת הפעולה מתבצע עדכון של שדה `createdAt` באמצעות התאריך והשעה הנוכחיים של יצירת הפוסט.

לאחר מכן המערכת מאתחלת את מספר הלייקים של הפוסט לערך 0, מכיוון שמדובר בפוסט חדש שעדיין לא קיבל לייקים.

לבסוף הפוסט נשמר במסד הנתונים MongoDB באמצעות `postRepository.save()`, והאובייקט שנשמר מוחזר מהפעולה.

סיבוכיות זמן ריצה:

$$O(1)$$

מלבד פעולת השמירה במסד הנתונים, הפעולה מבצעת מספר קבוע של פעולות בסיסיות.

שם הפעולה:

(`saveFileAsync`)

טענת כניסה:

הפעולה מקבלת `InputStream` של הקובץ שהועלה, שם קובץ מקורי, ואובייקט `Callback` מסוג `FileUploadCallback`.

טענת יציאה:

הפעולה אינה מחזירה ערך ישיר. במקום זאת, היא מפעילה `Callback`:

- `onSuccess(filePath)` במקרה שהקובץ נשמר בהצלחה
- `onError(errorMessage)` במקרה שהתרחשה שגיאה

תפקיד הפעולה:

הפעולה אחראית על שמירת תמונה שהועלתה למערכת בצורה אסינכרונית. שמירת הקובץ מתבצעת כ-`Long Task` בתוך `Thread` נפרד, כדי שלא לחסום את ממשק המשתמש בזמן שהקובץ נשמר.

קוד ממשק ה-`Callback`

```
public interface FileUploadCallback{
    void onSuccess(String filePath);
    void onError(String errorMessage);
}
```

קוד הפעולה

```
public void saveFileAsync(
    InputStream inputStream,
    String originalFileName,
    FileUploadCallback callback) {
    new Thread(() -> {
        try {
            Thread.sleep(1500);
            File uploadDir = new File("uploads");
```

```

    if (!uploadDir.exists()) {
        uploadDir.mkdirs();
    }

    String fileName =
        System.currentTimeMillis() + "_" + originalFileName;

    File file = new File(uploadDir, fileName);

    try (FileOutputStream fos = new FileOutputStream(file)) {
        inputStream.transferTo(fos);
    }

    String savedPath = "/uploads/" + fileName;

    callback.onSuccess(savedPath);
} catch (IOException e) {
    callback.onError("File upload failed.");
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
    callback.onError("File upload was interrupted.");
}
}).start();
}

```

הפעולה `saveFileAsync()` מבצעת שמירת קובץ שהועלה על ידי המשתמש. במקום לבצע את השמירה באופן רגיל ולחסום את הממשק, הפעולה יוצרת `Thread` חדש. בתוך `Thread` זה מתבצעת פעולת השמירה של הקובץ. בתחילת הפעולה קיימת השהיה קצרה באמצעות `Thread.sleep()`, המדמה `Long Task`, כלומר פעולה שיכולה לקחת זמן, כגון שמירת קובץ גדול או עיבוד תמונה. לאחר מכן המערכת בודקת האם קיימת תיקיית `uploads`. אם התיקייה אינה קיימת, המערכת יוצרת אותה. בהמשך נוצר שם קובץ ייחודי באמצעות `System.currentTimeMillis()`, כדי למנוע מצב שבו שני קבצים בעלי אותו שם ידרסו אחד את השני. לאחר יצירת הקובץ, המערכת מעתיקה את תוכן הקובץ מתוך ה-`InputStream` אל תיקיית `uploads`.

אם השמירה הצליחה, הפעולה מפעילה את המתודה `onSuccess()` של ה-`Callback` ומעבירה אליה את הנתביב של הקובץ שנשמר.

אם התרחשה שגיאה במהלך השמירה, הפעולה מפעילה את `onError()` ומעבירה הודעת שגיאה מתאימה.

שימוש ב-`Callback` מאפשר להפריד בין ביצוע הפעולה הארוכה לבין התגובה שמתרחשת לאחר סיום הפעולה.

סיבוכיות זמן ריצה:

$O(n)$

כאשר n הוא גודל הקובץ שנשמר.

הסיבה לכך היא שהמערכת מעתיקה את תוכן הקובץ מה-`InputStream` אל הדיסק, ולכן זמן הפעולה תלוי בגודל הקובץ.

13. תיאור מסד הנתונים

בסיס הנתונים הוא אחד המרכיבים החשובים ביותר במערכת, משום שהוא אחראי על שמירת כלל המידע הדרוש לפעולת הרשת החברתית. בפרויקט זה נעשה שימוש ב-MongoDB Atlas, שהוא שירות ענן לניהול מסד נתונים מסוג NoSQL.

הבחירה ב-MongoDB Atlas נבעה מהצורך בגמישות גבוהה, עבודה נוחה עם אובייקטים מורכבים ותמיכה מלאה ב-JSON וב-Spring Data MongoDB.

13.1 סוג מסד הנתונים

המערכת משתמשת במסד נתונים מסוג NoSQL המבוסס על מסמכים (Document-Oriented Database).

במקום טבלאות ושורות כמו במסדי נתונים רלציוניים MongoDB, משתמש ב:

- `Collections` אוספים ()

- `Documents` מסמכים ()

כל מסמך נשמר בפורמט דמוי JSON הנקרא BSON.

13.2 מבנה בסיס הנתונים

במערכת קיימים שני אוספים מרכזיים:

תפקיד	Collection
שמירת משתמשים	users
שמירת פוסטים	posts

13.3 אוסף המשתמשים - users

אוסף זה שומר את כל נתוני המשתמשים במערכת.

שדות מרכזיים

תיאור	סוג נתון	שדה
מזהה ייחודי	ObjectId	_id
שם משתמש	String	username
כתובת דוא"ל	String	email
סיסמה	String	password
תחומי עניין	Array	interests
פוסטים שאהב	Array	likedPostIds
וקטור העדפות	Object	preferenceVector
תאריך יצירה	Date	createdAt

דוגמת מסמך משתמש

```
{
  "_id": "665e1234abc1234567890001",
  "username": "yehoshua",
```

```

"email": "user@example.com",
"password": "1234",
"interests": [
  "tank",
  "weathering"
],
"likedPostIds": [
  "665e9999abc1234567891111"
],
"preferenceVector": {
  "tank": 5,
  "weathering": 3,
  "painting": 1
},
"createdAt": "2026-05-01T10:15:00Z"
}

```

13.4 אוסף הפוסטים - posts

אוסף זה שומר את כל הפוסטים במערכת.

שדות מרכזיים

שדה	סוג נתון	תיאור
_id	ObjectId	מזהה פוסט
userId	String	מזהה יוצר הפוסט
caption	String	תיאור הפוסט
mediaUrl	String	כתובת תמונה
mediaType	String	סוג מדיה
tags	Array	תגיות
likesCount	Integer	מספר לייקים
createdAt	Date	תאריך יצירה

דוגמת מסמך פוסט

```

{
  "_id": "665e9999abc1234567891111",

```



```

"userId": "665e1234abc1234567890001",
"caption": "Sherman tank diorama",
"mediaUrl": "/uploads/sherman.jpg",
"mediaType": "image",
"tags": [
  "tank",
  "weathering",
  "painting",
  "green"
],
"likesCount": 12,
"createdAt": "2026-05-05T14:30:00Z"
}

```

13.5 קשרים בין האוספים

למרות שמדובר במסד נתונים NoSQL, קיימים קשרים לוגיים בין האוספים.

קשר משתמש-פוסט

כל פוסט כולל את השדה:

```
"userId": "665e1234abc1234567890001"
```

באמצעותו ניתן לזהות מי יצר את הפוסט.

קשר משתמש-לייקים

השדה `likedPostIds` שומר את מזהי הפוסטים שהמשתמש אהב.

כך ניתן:

- למנוע Like כפול.
- להסיר Like.
- לעדכן את Preference Vector.

13.6 Preference Vector

וקטור ההעדפות הוא אחד המרכיבים החשובים ביותר במערכת.

הוא נשמר בתוך מסמך המשתמש כמבנה מסוג Map.

דוגמה

```
{
  "tank": 5,
  "painting": 2,
  "weathering": 4
}
```

מטרת הווקטור

- שמירת העדפות המשתמש.
- חישוב ציון התאמה לפוסטים.
- מימוש אלגוריתם ה-CBF.
- כאשר המשתמש מבצע Like:
- ערכי התגיות עולים.
- כאשר המשתמש מבצע Unlike:
- ערכי התגיות יורדים.

13.7 אינדקסים

MongoDB יוצר אינדקס אוטומטי על `_id`.

בנוסף, מומלץ להשתמש באינדקסים נוספים:

שדה	מטרת האינדקס
username	התחברות מהירה
email	מניעת כפילויות
userId	שליפת פוסטים לפי משתמש
tags	חיפוש תגיות
createdAt	מיון כרונולוגי

13.8 פעולות CRUD

המערכת מבצעת את כל פעולות ה:CRUD-

פעולה	תיאור
Create	יצירת משתמשים ופוסטים
Read	שליפת נתונים
Update	Like, Unlike ועדכון פוסטים
Delete	מחיקת נתונים (עתידיה)

Spring Data MongoDB 13.9

הגישה למסד הנתונים מתבצעת באמצעות Spring Data MongoDB .

יתרונות

- פחות קוד גישה למסד הנתונים .
- אינטגרציה מלאה עם Spring Boot.
- תמיכה ב-CRUD-אוטומטי .
- המרה אוטומטית בין Objects ל Documents.

MongoRepository 13.10

ה Repositories-במערכת יורשים מ-MongoRepository .

דוגמה

```
public interface UserRepository extends MongoRepository<User, String>
```

גישה זו מאפשרת:

- save()
- findAll()
- findById()
- deleteById()

ללא כתיבת SQL .

13.11 אבטחת מסד הנתונים

MongoDB Atlas כולל:

- משתמשים והרשאות .
- אימות גישה .
- חיבור מאובטח .
- גיבויים אוטומטיים .

בנוסף, הגישה למסד הנתונים מתבצעת דרך Connection String מוגן.

13.12 גיבויים ושחזור

MongoDB Atlas מבצע גיבויים אוטומטיים המאפשרים:

- שחזור מידע .
- הגנה מפני אובדן נתונים .
- שמירה על יציבות המערכת .

13.13 יתרונות MongoDB בפרויקט

השימוש ב-MongoDB Atlas-התאים במיוחד לפרויקט זה משום שהוא:

- גמיש מאוד .
- מתאים ל JSON- .
- נוח לעבודה עם Java .
- תומך במבני נתונים מורכבים .
- מאפשר אחסון בענן .

13.14 סיכום

בסיס הנתונים של המערכת מבוסס על MongoDB Atlas ומורכב משני אוספים מרכזיים – users – ו-posts. מבנה זה מאפשר אחסון גמיש של משתמשים, פוסטים, תגיות ווקטורי העדפות, תוך תמיכה מלאה במנגנון ההמלצות מבוסס Content-Based Filtering השימוש ב-Spring ו-MongoDB Atlas. Data MongoDB אפשר בניית שכבת נתונים מודרנית, יציבה וניתנת להרחבה.

14. מדריך למשתמש

מערכת Gallery Miniatures פותחה כרשת חברתית ייעודית לחובבי מיניאטורות, דיורמות ופרויקטי צביעה.

המערכת פועלת דרך דפדפן אינטרנט ומאפשרת למשתמשים לצפות בפוסטים, להעלות תוכן חדש ולקבל פיד מותאם אישית.

14.1 כניסה למערכת

בעת פתיחת המערכת המשתמש מגיע למסך הבית (HomeView).

במסך זה ניתן:

- לצפות בפיד הציבורי
- לעבור למסך התחברות
- לעבור למסך הרשמה

בנוסף, מוצג תפקיד המשתמש הנוכחי במערכת:

GUEST •

USER •

14.2 הרשמה למערכת

על מנת ליצור משתמש חדש יש ללחוץ על כפתור Register.

במסך ההרשמה המשתמש נדרש להזין:

Username •

Email •

Password •

לאחר לחיצה על כפתור Register המערכת שומרת את המשתמש במסד הנתונים.

במקרה של הרשמה מוצלחת מוצגת הודעת Notification והמשתמש מועבר למסך ההתחברות.

14.3 התחברות למערכת

משתמש קיים יכול להתחבר באמצעות מסך Login.

המשתמש מזין:

• Username

• Password

לאחר התחברות מוצלחת המשתמש מועבר לפיד האישי שלו.

אם פרטי ההתחברות שגויים, המערכת מציגה הודעת שגיאה מתאימה..

14.4 צפייה בפיד

המערכת כוללת מסך Feed המציג פוסטים של משתמשים.

משתמש Guest:

• יכול לצפות בפיד הציבורי בלבד.

משתמש מחובר:

• מקבל פיד מותאם אישית לפי תחומי העניין שלו.

כל פוסט כולל:

• תמונה

• כותרת / תיאור

• תגיות

• מספר לייקים

משתמש מחובר יכול לבצע Like ו-Unlike לפוסטים..

14.5 יצירת פוסט חדש

משתמש מחובר יכול ליצור פוסט חדש באמצעות מסך Create Post.

במסך זה המשתמש מזין:

• Caption

• Tags

• Media Type

בנוסף, ניתן להעלות תמונה מהמחשב המקומי באמצעות רכיב Upload.

לאחר לחיצה על כפתור Post הפוסט נשמר במסד הנתונים ומוצג בפיד.

14.6 צפייה בפרופיל המשתמש

מסך Profile מציג את פרטי המשתמש ונתוני הפעילות שלו.

במסך ניתן לראות:

- Username

- Email

- תאריך יצירת החשבון

- מספר לייקים

- Preference Vector

- רשימת הפוסטים של המשתמש

Preference Vector מציג את תחומי העניין של המשתמש לפי תגיות ומשקלים.

Navigation Bar 14.7

בכל מסכי המערכת מופיע Navigation Bar עליון.

הסרגל מאפשר מעבר מהיר בין מסכי המערכת.

משתמש Guest רואה:

- Home

- Feed

- Login

- Register

משתמש מחובר רואה:

- Home

- Feed

- Create Post

- Profile

- Menu

- Logout

14.8 התנתקות מהמערכת

משתמש מחובר יכול להתנתק באמצעות כפתור Logout.

לאחר ההתנתקות:

- ה-Session נמחק
- המשתמש חוזר למצב Guest
- המערכת מחזירה את המשתמש למסך הבית

15. בדיקות והערכת המערכת

לאחר השלמת שלב הפיתוח, בוצעו בדיקות מקיפות שנועדו לוודא כי כל רכיבי המערכת פועלים בצורה תקינה ועומדים בדרישות שהוגדרו בשלבי האפיון והתכנון. הבדיקות כללו בדיקות פונקציונליות, בדיקות אינטגרציה, בדיקות מנגנון ההמלצות ובדיקות שימושיות.

מטרת הבדיקות הייתה לבדוק:

- תקינות פעולות המשתמש.
- שמירת נתונים במסד הנתונים.
- יציבות המערכת.
- תקינות מנגנון ההמלצות.
- תקינות העלאת התמונות.
- תקינות ה-REST API.

15.1 סביבת הבדיקות

הבדיקות בוצעו בסביבה הבאה:

פירוט	רכיב
Windows 10/11	מערכת הפעלה
Java 21	שפת תכנות
Spring Boot	Framework
Vaadin	UI Framework
MongoDB Atlas	מסד נתונים

פירוט	רכיב
Visual Studio Code	IDE
Google Chrome	דפדפן
Postman	כלי API

15.2 סוגי הבדיקות

בפרויקט בוצעו מספר סוגי בדיקות:

בדיקות פונקציונליות

בדיקות שנועדו לוודא שכל פונקציה במערכת פועלת כנדרש.

בדיקות אינטגרציה

בדיקות שבדקו את התקשורת בין:

- Vaadin
- Services
- MongoDB Atlas
- REST API

בדיקות שימושיות

בדיקות שבחנו האם המערכת פשוטה ונוחה לשימוש.

בדיקות מנגנון ההמלצות

בדיקות שנועדו לוודא שה CBF-פועל בצורה תקינה.

15.3 בדיקות הרשמה והתחברות

בדיקת הרשמה

פרמטר	ערך
פעולה	יצירת משתמש חדש
תוצאה צפויה	המשתמש נשמר במסד הנתונים
תוצאה בפועל	המשתמש נשמר בהצלחה

פרמטר	ערך
סטטוס	עבר
בדיקת התחברות	
פרמטר	ערך
פעולה	Login עם פרטים תקינים
תוצאה צפויה	מעבר לפיד
תוצאה בפועל	המעבר בוצע בהצלחה
סטטוס	עבר
בדיקת סיסמה שגויה	
פרמטר	ערך
פעולה	Login עם סיסמה שגויה
תוצאה צפויה	הודעת שגיאה
תוצאה בפועל	הודעת השגיאה הוצגה
סטטוס	עבר

15.4 בדיקות יצירת פוסטים

בדיקת יצירת פוסט

פרמטר	ערך
פעולה	יצירת Post חדש
תוצאה צפויה	הפוסט נשמר
תוצאה בפועל	הפוסט נשמר והוצג בפיד

פרמטר	ערך
סטטוס	עבר
בדיקת העלאת תמונה	
פרמטר	ערך
פעולה	Upload של תמונה
תוצאה צפויה	הקובץ נשמר ומוצג
תוצאה בפועל	התמונה הוצגה בהצלחה
סטטוס	עבר

15.5 בדיקות Like ו-Unlike

בדיקת Like	
פרמטר	ערך
פעולה	Like לפוסט
תוצאה צפויה	likesCount גדל
תוצאה בפועל	המונה עודכן
סטטוס	עבר
בדיקת Unlike	
פרמטר	ערך
פעולה	Remove Like
תוצאה צפויה	likesCount קטן
תוצאה בפועל	המונה עודכן
סטטוס	עבר

15.6 בדיקות Preference Vector

בוצעו בדיקות שמטרתן לבדוק את תקינות עדכון וקטור ההעדפות.

תרחיש בדיקה

המשתמש ביצע Like לפוסט עם התגיות:

```
["tank", "weathering", "green"]
```

תוצאה צפויה

```
{
  "tank": 1,
  "weathering": 1,
  "green": 1
}
```

תוצאה בפועל

וקטור ההעדפות עודכן בצורה תקינה.

15.7 בדיקות מנגנון ההמלצות

מטרת הבדיקה

לוודא שפוסטים רלוונטיים מופיעים גבוה יותר בפיד.

נתוני משתמש

```
{
  "tank": 5,
  "weathering": 4,
  "painting": 2
}
```

פוסט לדוגמה

```
["tank", "weathering"]
```

חישוב Score

• tank = 5

• weathering = 4

סה"כ Score = 9

תוצאה

הפוסט דורג גבוה יותר מפוסטים אחרים בעלי Score נמוך יותר.

15.8 בדיקות REST API

המערכת נבדקה באמצעות Postman .

Endpoints שנבדקו

תוצאה	Endpoint
JSON תקין	/api/users
JSON תקין	/api/posts
נתוני משתמש הוחזרו	/api/users/{id}
נתוני פוסט הוחזרו	/api/posts/{id}

15.9 בדיקות שימושיות

נבדקו מספר מאפייני שימושיות:

תוצאה	נושא
תקין	ניווט בין מסכים
טובה	בהירות הממשק
גבוהה	קלות שימוש
מודרני וברור	עיצוב
מהיר	זמן תגובה

15.10 תקלות שהתגלו במהלך הבדיקות

במהלך הפרויקט התגלו מספר בעיות:

פתרון	תקלה
טעינה מחדש של המשתמש	Session לא התעדכן

פתרון	תקלה
שימוש ב-WebConfig	תמונות לא הוצגו
שינוי הגדרות Upload	Upload רענן את הדף
הוספת @Route	onRoute

15.11 מגבלות הבדיקות

- הבדיקות בוצעו על מספר מוגבל של משתמשים .
- לא בוצעו בדיקות עומס .
- לא בוצעו בדיקות אבטחה מתקדמות .
- הבדיקות בוצעו בסביבה מקומית בלבד .

15.12 הערכת המערכת

לאחר הבדיקות ניתן לקבוע כי:

- המערכת יציבה .
- הפונקציות המרכזיות פועלות בצורה תקינה .
- מנגנון ההמלצות פועל בהתאם לתכנון .
- ממשק המשתמש נוח לשימוש .
- החיבור למסד הנתונים אמין .

15.13 סיכום

שלב הבדיקות הוכיח כי המערכת עומדת בדרישות שהוגדרו בתחילת הפרויקט. כל הרכיבים המרכזיים – החל ממערכת ההתחברות ועד למנגנון ההמלצות – פעלו בצורה תקינה. הבדיקות הדגישו שהמערכת מסוגלת לספק חוויית שימוש יציבה ומותאמת אישית, תוך שמירה אמינה של הנתונים והצגת פיד חכם המבוסס על העדפות המשתמש.

16. מסקנות

במהלך פרויקט זה פותחה מערכת רשת חברתית ייעודית לחובבי מיניאטורות ודיורמות, הכוללת מנגנון המלצות חכם המבוסס על Content-Based Filtering (CBF) הפרויקט שילב בין פיתוח Web מודרני, עבודה עם מסד נתונים בענן, ניהול משתמשים ופיתוח אלגוריתם המלצות מותאם אישית. לאחר סיום שלבי התכנון, המימוש והבדיקות ניתן להסיק מספר מסקנות מרכזיות.

16.1 עמידה במטרות הפרויקט

המערכת עמדה בהצלחה במטרות שהוגדרו בתחילת העבודה:

- פותחה רשת חברתית מלאה מבוססת Web .
 - מומש מנגנון המלצות חכם .
 - בוצע חיבור למסד נתונים בענן .
 - נוספה אפשרות להעלאת תמונות .
 - פותח REST API .
 - נבנה ממשק משתמש מודרני באמצעות Vaadin .
 - בוצעה הפרדה לארכיטקטורת שכבות מסודרת .
- בנוסף, המערכת הצליחה להציג פיד מותאם אישית בהתאם להעדפות המשתמש, ובכך הודגם עקרון הפעולה של Content-Based Filtering בצורה מעשית.

16.2 יעילות מנגנון ההמלצות

מנגנון ההמלצות התבסס על תגיות (Tags) ו Preference Vector אישי לכל משתמש.

נמצא כי:

- ביצוע Like משפיע ישירות על ההעדפות .
 - הפיד משתנה בהתאם להתנהגות המשתמש .
 - פוסטים רלוונטיים מקבלים ציון גבוה יותר .
 - האלגוריתם פשוט יחסית למימוש אך עדיין מספק התאמה אישית יעילה .
- בנוסף, השימוש ב Preference Vector אפשר להבין בצורה ברורה כיצד המערכת "לומדת" את תחומי העניין של המשתמש.

16.3 יתרונות השימוש בטכנולוגיות שנבחרו

Spring Boot

Spring Boot אפשר:

- פיתוח מהיר.
- Dependency Injection.
- REST API מובנה.
- אינטגרציה נוחה עם MongoDB.

Vaadin

Vaadin אפשר:

- פיתוח מלא ב Java.
- יצירת ממשק משתמש מודרני.
- אינטגרציה פשוטה עם Spring Boot.
- חיסכון בצורך ב JavaScript.

MongoDB Atlas

MongoDB Atlas סיפק:

- אחסון בענן.
- גמישות גבוהה.
- עבודה נוחה עם JSON.
- שמירה פשוטה של Lists ו-Maps.

16.4 חשיבות ארכיטקטורת השכבות

חלוקת המערכת לשכבות:

- UI

- Services
- Repositories
- Database

תרמה ל:

- קוד מסודר וקריא .
 - תחזוקה קלה .
 - הרחבה עתידית פשוטה יותר .
 - הפרדת אחריות ברורה .
- גישה זו אפשרה פיתוח מקצועי יותר של המערכת.

16.5 אתגרים עיקריים במהלך הפיתוח

במהלך העבודה התגלו מספר אתגרים:

- ניהול Session של המשתמש המחובר .
 - עדכון Preference Vector בזמן אמת .
 - הצגת תמונות שהועלו למערכת .
 - מימוש Like ו Unlike-
 - ניווט בין מסכים ב Vaadin-
 - חיבור למסד נתונים בענן .
- פתרון בעיות אלו תרם רבות להבנה מעשית של פיתוח מערכות Web מודרניות.

16.6 ידע וניסיון שנרכשו

הפרויקט תרם להעמקת הידע בתחומים רבים:

- Java Object-Oriented Programming
- Spring Boot
- Vaadin
- MongoDB Atlas
- REST API
- Content-Based Filtering

- Web Development
- Software Architecture
- Client-Server Systems

בנוסף, הפרויקט סיפק ניסיון מעשי בעבודה על מערכת מלאה – החל משלב הרעיון ועד למערכת מתפקדת.

16.7 תרומת הפרויקט

הפרויקט מדגים כיצד ניתן לבנות רשת חברתית חכמה המשלבת:

- אחסון ענן .
- מנגנון המלצות .
- ממשק Web מודרני .
- ניהול משתמשים .
- העלאת קבצים .

המערכת מהווה בסיס טוב להמשך פיתוח ואף יכולה לשמש כבסיס לפרויקטים גדולים ומתקדמים יותר בעתיד.

16.8 סיכום

לסיכום, הפרויקט הצליח לממש את כל היעדים המרכזיים שהוגדרו בתחילת הדרך. המערכת שנבנתה היא רשת חברתית מלאה ומתפקדת, הכוללת מנגנון המלצות מותאם אישית המבוסס על Content-Based Filtering. העבודה שילבה ידע תיאורטי ומעשי מתחומי הנדסת התוכנה, פיתוח Web ומערכות המלצה, והיוותה חוויה לימודית משמעותית ומעשירה.

17. פיתוחים עתידיים

למרות שהמערכת שפותחה במסגרת הפרויקט מספקת רשת חברתית מלאה ומתפקדת עם מנגנון המלצות מבוסס Content-Based Filtering, קיימות אפשרויות רבות להרחבה ולשיפור בעתיד. הארכיטקטורה המודולרית שנבחרה מאפשרת להוסיף פונקציות חדשות ולשדרג את יכולות המערכת בצורה יחסית פשוטה.

פיתוחים עתידיים אלו יכולים להפוך את המערכת למתקדמת, חכמה ומציאותית יותר, בדומה לרשתות חברתיות מסחריות גדולות.

17.1 שיפור מנגנון ההמלצות

מנגנון ההמלצות הנוכחי מבוסס על תגיות בלבד. בעתיד ניתן לשלב אלגוריתמים מתקדמים יותר.

Collaborative Filtering

שיטה זו מבוססת על דמיון בין משתמשים ולא רק על תגיות.

יתרונות:

- גילוי תכנים חדשים.
- המלצות מגוונות יותר.
- התאמה טובה יותר למשתמשים פעילים.

Hybrid Recommendation System

שילוב בין:

- Content-Based Filtering
- Collaborative Filtering

גישה זו יכולה לשפר משמעותית את איכות ההמלצות.

Machine Learning

בעתיד ניתן לשלב מודלים של בינה מלאכותית ולמידת מכונה לצורך:

- ניתוח התנהגות משתמשים.
- חיזוי תחומי עניין.
- ניתוח תמונות.
- יצירת המלצות מדויקות יותר.

17.2 מערכת תגובות

כיום המשתמשים יכולים לבצע Like בלבד.

פיתוח עתידי אפשרי:

- הוספת תגובות לפוסטים.
- תגובות מקוננות.

- התראות על תגובות חדשות .
- מערכת זו תהפוך את הרשת החברתית לאינטראקטיבית יותר.

17.3 ממערכת Followers

ניתן להוסיף אפשרות לעקוב אחר משתמשים אחרים.

יכולות אפשריות

- Follow / Unfollow
- רשימת Followers
- רשימת Following
- פיד מבוסס משתמשים נעקבים

17.4 הודעות פרטיות

פיתוח מערכת Chat פנימית יאפשר למשתמשים לתקשר ביניהם.

אפשרויות עתידיות

- הודעות פרטיות .
- שיחות קבוצתיות .
- שליחת תמונות .
- התראות בזמן אמת .

17.5 אפליקציה סולרית

המערכת פועלת כיום דרך דפדפן בלבד.

בעתיד ניתן לפתח:

- אפליקציית Android.
- אפליקציית iOS.

באמצעות ה REST API-שכבר פותח, ניתן לחבר בקלות אפליקציה סולרית לשרת הקיים.

17.6 שיפור אבטחת המערכת

כיום המערכת מיועדת בעיקר ללמידה ולהדגמה.

בעתיד ניתן להוסיף:

- הצפנת סיסמאות באמצעות BCrypt.
- Authentication Tokens.
- JWT Authentication.
- HTTPS מלא.
- הרשאות משתמשים.
- מניעת מתקפות Injection.

17.7 אחסון תמונות בענן

כיום התמונות נשמרות מקומית בתיקיית uploads.

בעתיד ניתן להשתמש בשירותי ענן כגון:

- Amazon S3
- Cloudinary

יתרונות:

- מדרגיות גבוהה.
- גישה מהירה יותר.
- חיסכון במקום אחסון מקומי.

17.8 מערכת חיפוש

פיתוח עתידי נוסף הוא מנוע חיפוש מתקדם.

אפשרויות

- חיפוש לפי תגיות.
- חיפוש לפי משתמש.
- חיפוש לפי Caption.

- סינון לפי צבעים או קטגוריות .

17.9 מערכת ניהול (Admin Panel)

בעתיד ניתן להוסיף ממשק מנהל מערכת.

תפקידים אפשריים

- מחיקת פוסטים .
- חסימת משתמשים .
- ניהול תוכן .
- צפייה בסטטיסטיקות מערכת .

17.10 שיפור חוויית המשתמש

ניתן לשפר את ממשק המשתמש באמצעות:

- Dark Mode
- אנימציות
- Infinite Scrolling
- Notifications בזמן אמת
- Responsive Design מתקדם

17.11 פריסה לשרת ציבורי

כיום המערכת פועלת בסביבה מקומית.

בעתיד ניתן לפרוס אותה לשרת ציבורי באמצעות:

- Render
- Railway
- Heroku

• AWS

כך משתמשים יוכלו לגשת למערכת מכל מקום דרך האינטרנט.

17.12 סיכום

למערכת שפותחה קיימות אפשרויות הרחבה רבות, החל משיפור מנגנון ההמלצות ועד להפיכתה לרשת חברתית מלאה בקנה מידה גדול. הארכיטקטורה המודולרית והטכנולוגיות שנבחרו מאפשרות להוסיף בקלות פונקציות חדשות ולשדרג את יכולות המערכת בעתיד. פיתוחים אלו יכולים לשפר את חוויית המשתמש, את ביצועי המערכת ואת איכות ההמלצות, ולהפוך את הפרויקט לבסיס למערכת מקצועית ומתקדמת יותר.

18. ביבליוגרפיה

להלן רשימת המקורות ששימשו במהלך המחקר, התכנון, הפיתוח והמימוש של הפרויקט.

ספרים מקצועיים

1. Recommender Systems Handbook, Springer.

ספר מקצועי בתחום מערכות ההמלצה הכולל הסברים על Content-Based Filtering, Collaborative Filtering ו-Hybrid Systems.

<https://link.springer.com/book/10.1007/978-1-0716-2197-4>

2. Clean Code: A Handbook of Agile Software Craftsmanship, Prentice Hall, 2008.

ספר בנושא כתיבת קוד איכותי, קריא ותחזוקתי.

<https://www.oreilly.com/library/view/clean-code-a/9780136083238>

3. Programming: Principles and Practice Using C++, Bjarne Stroustrup.

ספר ללימוד עקרונות תכנות, מבני תוכנה ותכנות מונחה עצמים.

<https://www.stroustrup.com/PPP3.html>

4. Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley, 1994.

ספר מרכזי בנושא תבניות תכנון בתוכנה.

<https://www.oreilly.com/library/view/design-patterns-elements/0201633612>

תיעוד רשמי וטכנולוגיות

[Java Documentation](#) .5

תיעוד רשמי של Java.

[Spring Boot Documentation](#) .6

תיעוד רשמי של Spring Boot.

[Spring Data MongoDB Documentation](#) .7

תיעוד רשמי של Spring Data MongoDB.

[Vaadin Documentation](#) .8

תיעוד רשמי של Vaadin.

[MongoDB Documentation](#) .9

תיעוד רשמי של MongoDB.

[MongoDB Atlas Documentation](#) .10

תיעוד רשמי של MongoDB Atlas.

[Apache Maven Documentation](#) .11

תיעוד רשמי של Maven.

כלי פיתוח ובדיקות

[Visual Studio Code](#) .12

סביבת הפיתוח ששימשה לכתובת הקוד .

[Postman](#) .13

כלי לבדיקת REST API.

[MongoDB Compass](#) .14

כלי לניהול ובדיקת מסד הנתונים .

[Google Chrome](#) .15

דפדפן ששימש להרצת המערכת ולבדיקות .

מקורות אינטרנט נוספים

[Baeldung](#) .16

אתר מקצועי הכולל מדריכים בנושאי Spring Boot, MongoDB ו-Java.

[Stack Overflow](#) .17

קהילת מפתחים שסייעה בפתרון בעיות במהלך הפיתוח .

[W3Schools](#).18

מקור עזר ללימוד טכנולוגיות Web.

[MDN Web Docs](#).19

תיעוד בנושאי HTTP, HTTPS, TCP/IP ו-WebSocket.

סיכום

המקורות המפורטים לעיל שימשו בסיס תיאורטי ומעשי לפיתוח הפרויקט. שילוב בין ספרות מקצועית, תיעוד רשמי וכלי פיתוח מודרניים אפשר בניית מערכת מלאה, יציבה ומתקדמת, תוך יישום עקרונות הנדסת תוכנה ומערכות המלצה.